HAETAE Top-Down EasyCrypt Research Development Notes

Codex Week 1–16 Sprint Record

August 12, 2026

Abstract

These notes are research development notes, not a completed paper. They record the top-down proof architecture, the exact theorem statements attempted during the first fifteen sprints, the compiled EasyCrypt evidence, and the remaining named obligations that still block a public-API EUF-CMA theorem for the pinned HAETAE Jasmin implementation. Week 1 and Week 2 entries are retained as retrospective research records; Week 3 adds the exact generated raw-top mu control closure and procedure-level API copy-helper evidence. Week 4 records the raw ABI caller extraction, canonical address bridge, stores-free memory adapter, and the actual-caller residuals that keep the parent claim partial. Week 5 closes the sequential KeyGen export and accepted Verify control/input binding while retaining direct caller-observed mu equality and cross-call memory framing as explicit residuals. Week 6 closes the actual Sign output frame and region-local generated mu relation, adds an exact sequential Sign/Verify trace, and isolates the still-open product/replay step needed for direct equality of the traces' stored mu observations. Week 7 time-boxes that replay step into an explicit paper-boundary decision and shifts the main effort to mode-2 signature codec functional correctness, including a compiled actual prefix pack/unpack round trip. Week 8 closes the actual HBZ coefficient leaf inverse, concrete mode-2 table certificate, and focused/full extraction identity for the HBZ wrappers. Week 9 adds a common normalization-byte trace, closes the actual suffix-copy procedure, and constructs a unary harness using the actual encoder, copy, and decoder, while retaining both generated-loop trace refinements as explicit gates to the full core inverse. Week 10 isolates the encoder, proves the actual nested normalization byte/frame transition and actual success-size range, and records the still-open tail-aware outer trace relation without beginning decoder semantics. Week 11 replaces the local inner snapshot with an initial-array and pure-tail relation, connects generated table loads and final stores, and closes the actual encoder's success-conditioned full-suffix trace refinement. Week 12 closes the exact-trace semantic refinement of the actual generated decoder, including symbol recovery, exact byte consumption, final-state checking, and the output-tail frame, while leaving actual harness composition and termination as explicit downstream obligations. Week 13 composes the actual encoder, suffix-copy helper, and decoder in one unary harness, deriving the decoder's pointwise trace reads from the actual encoder/copy post-state and closing the rANS core inverse as success-conditioned partial correctness. Week 14 lifts the completed leaf and rANS results through exact internal procedure boundaries, proves the actual full encoder and decoder wrappers, composes their visible zero-size failure and nonzero success branches, and formally transfers the result to the SignaturePack/Unpack extraction boundary. Week 15 specializes the actual encoder to the zero-HBZ/all-six input, proves a 1020-byte coarse normalization budget, connects actual failure to budget overflow, excludes that failure in Hoare partial correctness, and lifts the fixed-input failure-exclusion result through the full HBZ and production pack/unpack boundaries while leaving termination explicit and open. Week 16 directly composes the actual M23 matrix and finalizer helpers, proves their exact and semantic snapshot consequences including the coefficient low/high split and a snapshot-only congruence modulo $2q$, and records the missing full-NTT-to-security-model multiplication bridge as the exact blocker to the faithful paper KeyGen equation. The KG–NTT–MUL continuation fresh-compiles the last honest NTT-word representation edge, identifies the missing

odd-root orthogonality/convolution and array-to-list adapter leaves, and freezes KeyGen at KG-2/finalization as STOP-KG-NTT. The next Sign pass directly composes the three actual accepted-core helpers and proves their branch control, but freezes S-1–S-7 as STOP-SIGN-CHAL-MODE2 because the full highbits/LSB/μ challenge semantics leaf is absent. The active MINCORE lane moves to Verify.

Contents

1 Scope and Proof Architecture

These notes cover HAETAE mode 2 only. The official specification source is the pinned PDF artifact [1]. The implementation target is the pinned Jasmin source tree [3]. Existing EasyCrypt artifacts are reused only through explicit imports with premise preservation [2].

The top-down structure used in this workspace is:

$$\mathsf{prefix}_{992}(sk) = vk \Rightarrow \mathsf{equal\_hash\_inputs} \Rightarrow \mathsf{take}_{32}(\mu^{64}_{\mathsf{sign}}) = \mu^{32}_{\mathsf{verify}} \Rightarrow \mathsf{equal\_challenge\_suffix}.$$

The Week 1 and Week 2 seams are intentionally narrower than full API correctness. The work records exact theorem statements, premises, and blocked obligations instead of hiding them behind new axioms.

Current theorem-graph position.

- `KG-PREFIX-CALL`: PROVED as generated-procedure partial correctness; retry termination remains separate.
- `MU32-HASH-RAW-HELPERS`: PROVED through the actual generated absorb, Keccak, finalize, and squeeze helpers.
- `MU32-HASH`: PARTIAL until the exact generated top-level control adapter is proved.
- `MU-CHAL-COMPOSITION`: PROVED with local $\delta_\mu = 0$ for the explicit raw wrappers and challenge suffix.

Week 1 note status. Section 2 is a retrospective reconstruction from the Week 1 artifacts already checked in this workspace. It is not a contemporaneous lab notebook.

2 Week 1 Transcript Seam

Week 1 established the first extracted cross-module theorem on the challenge suffix path:

$$\mathsf{take}_{32}(\mu^{64}_{\mathsf{sign}}) = \mu^{32}_{\mathsf{verify}} \Longrightarrow \mathsf{equal\_challenge\_suffix}.$$

Primary claim IDs.

- `MU32-ABSORB`: PROVED.
- `CHAL-LIST`: PROVED.
- `CHAL-CALL`: PARTIAL.

Pinned procedures and theories.

- Sign procedure: `__sign\_challenge\_shake256\_absorb\_mu32`.
- Verify procedure: `__verify\_shake256\_absorb\_mu32`.
- EasyCrypt theorem: `ExtractedChallengeAbsorb.sign\_verify\_mu32\_absorb\_from\_pos64`.
- EasyCrypt theorem: `TranscriptBytes.challenge\_input\_eq\_components`.

Proof strategy. The extracted theorem fixed sponge position 64 and required an explicit `mu32_prefix` predicate over generated arrays. A separate witness lemma showed that this premise is satisfiable, which ruled out one vacuity risk but did not yet connect the premise to actual KeyGen/Sign/Verify call paths.

Verification evidence. Week 1 used fresh `-no-eco` compilation and regenerated focused extractions. The operational source of truth remains the Markdown ledger and the verifier logs under `haetae-topdown-easycrypt/logs/`.

3 Week 2 Packed-Key Prefix

Research question and claim. `KG-PREFIX-CALL` asks whether the actual generated mode-2 packer and KeyGen caller establish

$$\mathsf{prefix}_{992}(sk) = vk.$$

The corresponding paper algorithm is KeyGen in Figure 8 of the pinned specification [1]. The implementation source is `keypair.jazz`, SHA-256 `8b2ddac2862122d1c9d58767cbc5caa2caad39c8f41fa8096d`

Procedures and generated source. The proof opens the generated procedures `\_pack\_sk\_m23`, `\_pack\_vec\_eta\_to`, and `\_pack\_vec2\_eta\_to`. It then lifts their result through `\_keypair\_full\_m2` and `crypto\_sign\_keypair\_internal\_mode2\_jazz`. The imported parent extraction has SHA-256 `248f8157e348e3f294665d12573a58ee58e860884356bfe82324fda64de5d0b4`. The independently regenerated packer-focused target has SHA-256 `7062715ca8fe449f15632c315cafdb316bddad6c39`

Compiled statements. Theory `ExtractedPackedKeyPrefix` contains:

```
hoare [Parent._pack_sk_m23 :
  vkp = vk0 /\ vkbytes = W64.of_int 992 /\
  mcount = W64.of_int 3 /\ kcount = W64.of_int 2
  ==> vk_prefix_eq res vk0 992]
```

under theorem name `pack\_sk\_m23\_mode2\_vk\_prefix`. It also proves:

```
hoare [Parent._keypair_full_m23 :
  k = 2 /\ m = 3 /\ vkbytes = 992
  ==> vk_prefix_eq res.`2 res.`1 992]
```

as `keypair\_full\_m23\_mode2\_return\_prefix`, followed by an unconditional mode-2 internal-entry lift `keypair\_internal\_mode2\_return\_prefix`.

Proof strategy and invariant. The initial copy loop maintains

$$\mathsf{copied\_prefix}(skp, vk_0, \mathsf{to\_uint}(i)).$$

The later-write frame is split rather than assumed:

- `pack\_vec\_eta\_to\_prefix\_frame` proves that the $3 \cdot 64$ byte eta-vector region beginning at offset 992 preserves the prefix;
- `pack\_vec2\_eta\_to\_prefix\_frame` does the same for the $2 \cdot 96$ byte eta2 region; and
- `mode2\_key\_index\_after\_prefix` proves that the final 32-byte key write also begins after the prefix.

Reachability and non-vacuity. This is a Hoare partial-correctness result. It has no hidden termination premise: every execution that returns satisfies the prefix relation. Losslessness of the retry loop, the probability of termination, and the output distribution remain separate obligations. The returned relation is consumed constructively in `Mode2MuChallengeComposition.keygen\_prefix` it is not merely witnessed by arbitrary arrays.

Spec/implementation difference. The paper presents structured keys, whereas the Jasmin code lays the packed VK at the beginning of a 2752-byte SK and appends eta vectors and a 32-byte key. The theorem validates this concrete layout relation only; it does not yet prove canonical parsing or equality to the paper's full key distribution.

Verification and status. `KG-PREFIX-CALL` is PROVED as partial correctness. The target fresh-compiles with `-no-eco`; packer regeneration and its output hash are checked by `scripts/verify-all.sh`. The next minimum KeyGen obligations are `OBL-KG-TERMINATION` and the public-memory lift, not another array-prefix lemma.

4 Week 2 μ Hash Prefix

Research question. For the internal/raw-pre branch, do the actual generated hash helpers produce

$$\mathsf{take}_{32}(\mu^{64}_{\mathsf{sign}}) = \mu^{32}_{\mathsf{verify}}$$

when they receive the same 992 VK bytes, pre bytes, message bytes, and lengths? This is the byte-level implementation instance of the H_{gen} use in Sign and Verify in Figure 8 [1].

Sources and focused targets. The Sign root is `\_sf\_mu\_rawpre` in `sign.jazz`, SHA-256 `fd58271f3ada888dfcef4bcf89027c986cc91464a7cb8446b6de11e40037ac31`. The Verify root is `\_\_verify\_hash\_mu` in `verification\_transcript.jazz`, SHA-256 `c2a9865274c5bf2ecbc2ccf6d53065`. The regenerated generated-theory hashes are `c2232e64...e83aa7` for Sign and `fcee969a...c0ebb7` for Verify; the full values are pinned in the extraction manifest.

Compiled generated-procedure lemmas. Theory `ExtractedMuHashPrefix` proves:

- `sign\_verify\_keccak\_init\_from\_same\_state`;
- `sign\_verify\_keccakf1600\_from\_same\_state`;
- `sign\_verify\_finalize\_from\_same\_state`;
- `sign\_verify\_absorb\_addr\_from\_same\_state`;
- `sign\_sk\_verify\_vk\_absorb\_mode2`; and
- `sign\_verify\_final\_squeeze\_mu32\_prefix`.

The last two are the nontrivial representation bridges. The first relates a `BArray2752` SK prefix to Verify's raw-memory loads for exactly 992 bytes. The second relates Sign's word-oriented `\_sf\_shake256\_squeeze64` to Verify's byte-oriented `\_\_poly\_challenge\_squeeze256\_32`, including little-endian byte order and the actual Keccak calls.

Composed theorem and premises. `sign\_verify\_raw\_mu\_top\_wrappers\_prefix` composes the actual helpers. Its premises are:

$$\begin{aligned}
&\mathsf{sk_memory_prefix}(sk_0, mem_0, base),\\
&\mathsf{to_uint}(base) + 992 < 2^{64},\\
&preaddr_{\mathrm{S}} = prep_{\mathrm{V}}, \quad prelen_{\mathrm{S}} = prelen_{\mathrm{V}},\\
&maddr_{\mathrm{S}} = mp_{\mathrm{V}}, \quad mlen_{\mathrm{S}} = mlen_{\mathrm{V}},\\
&\mathsf{Glob.mem}_{\mathrm{S}} = \mathsf{Glob.mem}_{\mathrm{V}} = mem_0.
\end{aligned}$$

Its postcondition is bytewise equality for every $0 \le i < 32$. The proof uses no abstract SHAKE-output-prefix axiom. The word/byte invariant uses the pinned lemmas `drop\_bytes0`, `drop\_bytes\_succ`, and `rate\_lane\_byte\_get8`, with every range premise discharged.

Exact call-site boundary. The local modules `SignRawMuTop` and `VerifyRawMuTop` reproduce initialization and the complete generated helper sequence. An exact refinement from the generated `\_sf\_mu\_rawpre` and the raw branch of `\_\_verify\_hash\_mu` to these wrappers has not yet compiled. The residual is `OBL-MU-TOPLEVEL-CONTROL`: generated branch condition, local-variable scheduling, argument binding, and pointer-protection premises. Thus `MU32-HASH-RAW-HELPERS` is PROVED, while its parent `MU32-HASH` remains PARTIAL.

Public-context path. The context variant is kept separate: Sign uses `\_sf\_prepare\_pre\_raw` followed by `\_sf\_mu\_preptr`, whereas Verify uses the high-bit branch. The next statement must require $ctxlen \leq 255$ and prove that the encoded pre is exactly the length byte followed by ctx. This variant is SPECIFIED; no raw-path theorem is silently generalized to it.

Verification. All listed helper/composition lemmas fresh-compile with `-no-eco`. Focused extraction hashes and the no-axiom/no-hole scans are checked by `scripts/verify-all.sh`.

5 Composition and Security Boundary

Target chain. The first seam is:

$$\mathsf{prefix}_{992}(sk) = vk \Rightarrow \mathsf{equal\_hash\_inputs} \Rightarrow \mathsf{take}_{32}(\mu^{64}_{\mathsf{sign}}) = \mu^{32}_{\mathsf{verify}} \Rightarrow \mathsf{equal\_challenge\_suffix}.$$

Reachability from KeyGen. `keygen\_prefix\_reaches\_mu\_memory` consumes the actual `vk\_prefix\_eq` returned by the generated KeyGen theorem and constructs

$$mem' = \mathsf{stores}(mem, \mathsf{to\_uint}(base), \mathsf{take}_{992}(\mathsf{to\_list}(vk))).$$

It proves the exact `sk\_memory\_prefix` premise required by the mu hash theorem. The independent lemma `mode2\_base\_zero\_no\_wrap` proves that $base = 0$ satisfies the 992-byte address-range condition. Together these are a concrete non-vacuity/reachability argument, not an existential witness over unrelated mu arrays.

Security-facing composition theorem. `raw\_mu\_to\_challenge\_suffix\_zero\_loss` compares two composed programs:

SignRawMuTop; `\_\_sign\_challenge\_shake256\_absorb\_mu32`,
VerifyRawMuTop; `\_\_verify\_shake256\_absorb\_mu32`.

In addition to the raw mu premises of Section 4, it requires equal initial challenge states/pointers and position 64. Its postcondition is equality of the returned challenge state and position. The proof composes `sign\_verify\_raw\_mu\_top\_wrappers\_prefix` with the Week 1 theorem `sign\_verify\_mu32\_absorb\_from\_pos64`.

List-level adapter. `challenge\_input\_eq\_after\_raw\_mu` reuses `challenge\_input\_eq\_from\_mu\` together with `challenge\_input\_eq\_components`, this gives equality of the list transcript whenever highbits and LSB components are also equal.

Local loss. For the proved wrapper/helper seam,

$$\delta_\mu = 0.$$

This is deterministic equivalence, so no statistical bad event is introduced at this seam. The statement is deliberately not promoted to $\delta_{\mathsf{Sign}} = 0$, $\delta_{\mathsf{Verify}} = 0$, or $\delta_{\mathsf{Encoding}} = 0$. Exact generated top-level control, highbits/LSB packing, the full challenge call, public memory contracts, and paper-ROM correspondence remain open.

Top-down graph position. This theorem sits below the exact public APIs and above the Week 1 mu32 absorb lemma. Closing `OBL-MU-TOPLEVEL-CONTROL` will make the relation an actual raw call-site theorem; closing `OBL-HIGH-LSB-PACK` will then permit the complete challenge-input relation.

Verification and status. `MU-CHAL-COMPOSITION` is PROVED for the local raw wrappers. The target fresh-compiles with `-no-eco`; its concrete address witness and memory-construction lemmas compile in the same theory.

6 Failed Approaches and Open Obligations

Rejected or failed approaches.

- An abstract SHAKE64/SHAKE32 prefix axiom was rejected because it would hide permutation, padding/finalize, output byte order, and the two generated squeeze loops.
- The Week 1 satisfiable mu-array witness was not treated as call-site reachability. Week 2 instead derives the key/memory premise from a returned generated KeyGen pair.
- A direct `inline; sim` proof between `\_sf\_mu\_rawpre` and `\_\_verify\_hash\_mu` stalled on generated local/control scheduling, raw-branch selection, argument binding, and pointer-protection goals. Week 2 retained the strongest complete generated-helper theorem and named the residual `OBL-MU-TOPLEVEL-CONTROL`; Week 3 closed it with two exact-to-wrapper adapters and a transitivity proof.
- The raw and public-context paths were not merged because their branch conditions and encoded pre strings differ.
- KeyGen termination was not placed in the prefix theorem's premise. Doing so would obscure partial correctness versus losslessness.

Open implementation obligations.

- `OBL-API-KEY-MEMORY-RAW`: sequential export framing, word/integer caller address binding, heap ownership, and harmful aliasing.
- `OBL-MU32-HASH-CONTEXT`: high-bit branch and $ctxlen \parallel ctx$ encoding.
- `OBL-HIGH-LSB-PACK`: identical pre-mu challenge bytes.
- `OBL-FIPS202-TRANSCRIPT`: paper-level domain, padding, endianness, and ROM-query adapter.
- `OBL-KG-TERMINATION`: retry losslessness, separately from the proved prefix property.

Open cryptographic and numerical obligations. The accepted signing distribution, challenge support/cardinality/min-entropy, two-transcript extraction, exact MLWE/BST-MSIS handoffs, fixed-point FFT tail, and TieMin policy remain outside this seam. They were not advanced merely because deterministic helper equivalence now compiles.

Known spec/implementation differences. The paper uses structured key and transcript objects. Sign reads the VK from a packed `BArray2752` prefix, while Verify reads it through raw memory; Sign emits 64 mu bytes wordwise, while Verify emits 32 bytes bytewise; and public context is encoded through a separate branch. Week 3 proves the raw generated top relation and isolated API copy helpers, but not their whole public-call composition or the paper model.

Non-vacuity conclusion. All new implementation premises are either explicit or constructible. No new implementation axiom exists. The raw top-control obligation is now compiled; the remaining API-memory obligation names the concrete sequential frame and caller-binding steps rather than quantifying them away.

7 Next Work

Week 3 selection. The packed prefix closed, while the parent raw call-site μ-hash claim is still partial. The decision rule therefore selects another focused SHAKE/control/memory sprint:

1. prove an exact refinement between `\_sf\_mu\_rawpre`, the raw branch of `\_\_verify\_hash\_mu`, and the compiled raw wrappers;
2. lift the constructed 992-byte relation through actual Sign/Verify callers and public memory contracts; and
3. rerun extraction hashes, fresh EasyCrypt compilation, non-vacuity checks, baselines, and the LaTeX build.

Go/no-go. This is a *go* for the narrow control-flow and memory-lifting work. It is a *no-go* for widening the claimed public-API refinement or paper-level EUF-CMA result. Highbits/LSB packing and full challenge-call equivalence begin only after `OBL-MU-TOPLEVEL-CONTROL` closes.

Minimum next theorem. The next nontrivial deliverable must show that an actual terminating raw generated Sign/Verify call establishes the already-proved wrapper pre/post relation. It must expose branch and memory premises and must not introduce a SHAKE or implementation-semantic axiom.

Longer path. After the exact raw call site closes, the order is public-context encoding, highbits/LSB equality, full challenge/ROM query equivalence, then the broader Sign/Verify arithmetic and security-distribution obligations. KeyGen termination, FFT/TieMin, sampler/rejection distributions, and forking remain separate workstreams.

8 Week 3 Generated μ Control

Research question and claim. Week 3 targets `OBL-MU-TOPLEVEL-CONTROL`. The required statement is no longer about proof-only local wrappers; the actual generated procedures must appear directly in the theorem:

$$\mathsf{take}_{32}(\mathsf{sf\_mu\_rawpre}(sk, pre, m)) = \mathsf{verify\_hash\_mu}(vk, pre, m).$$

More precisely, the claim is

$$\mathsf{raw\_prelen}(\ell) \wedge \mathsf{prefix}_{992}(sk, mem, vk) \Longrightarrow \mathsf{take}_{32}(\mathsf{sf\_mu\_rawpre}(sk, pre, m)) = \mathsf{verify\_hash\_mu}(vk, pre, m).$$

Generated control sequence. The Sign side is the actual generated `SignMuHashTarget.M.\_sf\_mu\_rawp`
Its control sequence is:

1. stack/local initialization and zeroed Keccak state;
2. `\_keccak\_init\_state`;
3. `\_sf\_shake256\_absorb\_sk` with `vkbytes = 992`;
4. raw absorb of `pre`;
5. raw absorb of `message`;
6. `\_sf\_shake256\_finalize`;

7. one additional `\_keccakf1600`;
8. `protect\_ptr`;
9. `\_sf\_shake256\_squeeze64`.

The Verify side is the actual generated `VerifyMuHashTarget.M.\_\_verify\_hash\_mu`. After initialization and the 992-byte VK absorb, it computes the branch guard

$$ctxflag := prelen \gg 63.$$

The raw branch is taken only after this actual generated shift computes zero.

Why exact-to-wrapper refinement was chosen. The Week 2 helper theorem already proved the semantic core of the raw hash path, but only through local wrappers that reconstructed the top-level control shape. A direct `inline; sim` proof over the two generated procedures stalled on:

- generated local-variable scheduling;
- erased spill/unspill/declassify instructions;
- the Verify raw/context branch split; and
- mismatch between the generated tail-call boundary and the wrapper-level theorem boundary.

Week 3 therefore isolates two unary exact adapters and composes them by transitivity rather than attempting one monolithic whole-program simulation.

Compiled exact adapters. Theory `ExactMuTopControl` proves:

- `sf\_mu\_rawpre\_refines\_raw\_top`;
- `raw\_prelen\_shr63\_zero`;
- `raw\_prelen\_branch\_guard`; and
- `verify\_hash\_mu\_raw\_refines\_raw\_top`.

The Verify-side predicate is

$$\mathsf{raw\_prelen}(prelen) := 0 \leq \mathsf{to\_uint}(prelen) < 2^{63}.$$

The two branch lemmas prove that this predicate implies that the actual generated shift-by-63 word operation yields 0, so the raw branch is selected semantically rather than by assumption.

Actual top-level theorem. The closing theorem is `sign\_verify\_generated\_raw\_mu\_prefix`. Its left and right procedures are exactly:

```
Sign._sf_mu_rawpre
Verify.__verify_hash_mu
```

Its premises include:

- Sign `vkbytes = 992`;
- Verify `vklen = 992`;

- equal raw pre/message pointers and lengths;
- `raw\_prelen prelen`;
- a KeyGen-derived `sk\_memory\_prefix`; and
- the explicit VK, pre, and message no-wrap predicates, including $\mathsf{to\_uint}(base) + 992 < 2^{64}$.

Its conclusion is bytewise equality of the first 32 Sign output bytes and the entire Verify output.

Reachability and non-vacuity. The theorem is not justified by a generic witness over unrelated arrays. Theory `ExactMode2RawMuComposition` proves `generated\_raw\_mu\_preconditions\_from\_k` which packages:

- the returned KeyGen prefix theorem;
- the concrete witness $base = 0$;
- `raw\_prelen W64.zero`; and
- the constructed Verify memory image obtained by storing the packed VK.

It further proves the Hoare theorem `keypair\_internal\_return\_reaches\_generated\_raw\_mu\_precon` over the actual generated `crypto\_sign\_keypair\_internal\_mode2\_jazz`. Every terminating execution therefore supplies this constructed local-array/raw premise set; KeyGen termination is neither assumed nor concluded. Thus the actual generated theorem premises are satisfiable by a concrete call path.

Status. `OBL-MU-TOPLEVEL-CONTROL` is PROVED for the raw/internal path. The public-context branch remains separate and unproved.

9 Week 3 API Memory Reachability

Research question and claim. The strong secondary question is whether the packed-key relation used in the HAETAE KeyGen/Sign algorithms (official PDF, Figure 8) survives the actual Jasmin export/import loops. The whole claim is tracked as `OBL-API-KEY-MEMORY-RAW`:

$$\mathsf{KeyGen.local}\ (vk, sk) \implies \mathsf{memory}[vku, vku{+}992) = vk \wedge \mathsf{memory}[sku, sku{+}1408) = sk \implies \mathsf{prefix}_{992}(sk_{\mathrm{Sign}}) = v$$

Pinned implementation and extraction. Theory `ApiKeyMemoryBridge` opens the generated procedures `__kp_api_copy_2080_to_addr` and `__kp_api_copy_2752_to_addr` from `keypair.jazz`, plus the Sign and Verify copies of `_api_copy_raw_to_2752_prefix`. The source SHA-256 values are `8b2ddac2...3e01` for KeyGen, `fd58271f...ac31` for Sign, `789f36e7...b06` for Verify, and `4d04b8e7...a8a0` for their shared `api.jinc`. Regenerated theory hashes are recorded in `manifests/generated-extractions.sha256`; regeneration is part of the main verifier.

Compiled generated-helper theorems. The following actual procedure-level results fresh-compile:

- `keygen_export_vk_mode2_prefix`: under `valid_region_w64` and length 992, the VK exporter writes every byte in the external interval to the corresponding local VK byte;
- `keygen_export_sk_mode2_prefix`: the analogous statement for all 1408 mode-2 SK bytes;

- `keygen_export_sk_mode2_prefix_frames_vk`: under the explicit minimal `disjoint_regions` premise, the same actual SK exporter also preserves a previously established 992-byte VK memory prefix;
- `sign_import_mode2_sk_prefix`: under `valid_region_int` and length 1408, the actual Sign importer returns a local array whose first 992 bytes equal memory;
- `verify_import_mode2_vk_prefix`: the actual Verify importer returns the 992-byte VK prefix; and
- `sign_verify_api_importer_same_inputs`: identical memory, source, and length inputs produce identical results for the two generated importer copies.

These are Hoare partial-correctness or exact equivalence statements. They do not assume or conclude public API termination beyond the finite extracted loops, nor do they manufacture a desired memory with an existential store.

Proof strategy and loop invariant. Each exporter invariant records the processed counter interval and a `mem2080_prefix` or `mem2752_prefix` relation. The no-wrap premise justifies converting the generated word addition into integer address addition. Each importer first proves equality for indices below its requested length, then shows the zero-fill tail cannot modify indices below 992. This last frame fact is essential for Sign, whose mode-2 SK length is 1408 although the downstream hash reads only its 992-byte VK prefix.

Adapters into the mu theorem. The pure, compiled lemmas `exported_mode2_regions_agree`, `imported_prefixes_agree`, and `imported_sign_sk_reaches_mu_memory` compose the packed-key prefix with the helper postconditions and obtain exactly `ExtractedMuHashPrefix.sk_memory_prefix`. Thus the dependency graph has reached

$$\mathsf{prefix}_{992}(sk) = vk \rightarrow \mathsf{actual\ exports} \rightarrow \mathsf{actual\ imports} \rightarrow \mathsf{sk\_memory\_prefix},$$

but the arrows are not yet a single sequential public-call theorem.

Address-model difference and aliasing. The generated KeyGen exporters retain `W64.t` addresses. In contrast, the selected Sign/Verify helper parameters originate as Jasmin `ui64` but are emitted by `jasmin2ec` as mathematical `int`. The theory therefore keeps `valid_region_w64` and `valid_region_int` distinct; no axiom equates their addresses. Concrete zero/adjacent addresses witness both predicates and `disjoint_regions`, so the contracts are satisfiable.

Separation is correctness-critical when the later SK export must preserve the earlier VK export. It is not required merely to compare two isolated, read-only importer calls. A stronger all-buffer separation policy would be a proof convenience and has not been adopted. Time-dependent aliasing at the unextracted public callers remains unanalyzed.

Failed boundary and next minimum obligation. The first attempt to concatenate the two exporters in a proof-only pair wrapper became unstable at the intermediate global-memory ghost binding. Replacing that wrapper with a stronger postcondition on the actual SK exporter succeeded: the compiled frame theorem retains the earlier VK predicate bytewise. What remains cannot be solved by that local frame alone. The word and integer pointers still cannot be identified syntactically, and adding such an identification as an axiom was rejected. The next minimum theorem must compose the two actual exporter calls at their real caller, bind $\mathsf{to\_uint}(sku), \mathsf{to\_uint}(vku)$ to the two importer integer addresses, and carry the resulting memory through the public call sequence.

Verification and status. The focused extraction script and `easycrypt compile -script -no-eco` compile all results above; the combined command is `scripts/verify-all.sh`. The generated helper claims are PROVED as partial correctness. The final run reports 14 authored targets compiled with `-no-eco`, 20 selected baselines, a successful notes build, and unchanged read-only roots. The whole `OBL-API-KEY-MEMORY-RAW` is PARTIAL, with the exact residual being actual-caller export composition, address binding, and public-call orchestration.

10 Week 3 Results and Next Gate

Exact Week 3 result. Week 3 closes the first generated top-level seam:

$$\mathsf{prefix}_{992}(sk) = vk \Rightarrow \mathsf{equal\_hash\_inputs} \Rightarrow \mathsf{take}_{32}(\mu^{64}_{\mathsf{sign}}) = \mu^{32}_{\mathsf{verify}} \Rightarrow \mathsf{equal\_challenge\_suffix}.$$

The important difference from Week 2 is that the central theorem is now stated directly over the actual generated `\_sf\_mu\_rawpre` and `\_\_verify\_hash\_mu` procedures.

Security-facing boundary. Theory `ExactMode2RawMuComposition` then proves `generated\_raw\_mu\_to\` which invokes the actual generated raw hash procedures before the actual generated mu32 challenge absorb procedures. Therefore

$$\delta_{\mu,\mathrm{raw\_top}} = 0.$$

This bound is intentionally local. It does not imply:

- $\delta_{\mathsf{Sign}} = 0$;
- $\delta_{\mathsf{Verify}} = 0$;
- $\delta_{\mathsf{Encoding}} = 0$; or
- any statement about the public-context path.

Secondary procedure-level progress. The actual generated VK/SK exporters and Sign/Verify raw importers now have compiled prefix theorems in `ApiKeyMemoryBridge`. Their local adapters reach the same `sk_memory_prefix` predicate consumed by the generated mu theorem, and the actual SK exporter has a compiled disjoint-write frame preserving a prior VK region. This is stronger than the prior constructed-memory witness, but it is not yet a public-call proof.

Remaining named obligations. The next technical gate is not another wrapper theorem. It is the exact public-memory/copy reachability chain under `OBL-API-KEY-MEMORY-RAW`: actual-caller composition of the two exports, the `W64.t`-to-`int` caller binding, and call-site ownership/alias orchestration. After that, the next cryptographic refinement work is:

- exact highbits/LSB packed-byte equality;
- full challenge-call equivalence;
- byte-faithful FIPS202/ROM transcript correspondence.

Failed approach retained as evidence. The rejected monolithic `inline; sim` strategy remains important negative evidence. The difficulty was not the helper semantics, which were already proved, but the generated top-level control schedule and branch/tail alignment. This justifies the exact-to-wrapper decomposition as a method choice rather than a convenience shortcut.

Go / no-go rule. The evidence-based Week 4 gate is:

- if the composed public copy-helper chain closes, proceed to highbits/LSB and the full challenge-call equivalence;
- because Week 3 closes the raw top-level seam and individual copy helpers but not their public composition, remain focused on API caller/address/alias reachability;
- if the exact generated top theorem were to regress, stop widening the proof surface and redesign the program-equivalence boundary instead of adding more wrappers.

11 Week 4: raw ABI extraction and address correspondence

This chapter is a research-development record, not a completed public-API correctness proof. The Week 4 question is whether external buffers used by the three raw/internal ABI procedures supply the memory premise of the already proved generated mu theorem without constructing an artificial `stores mem 0` witness.

11.1 Actual extraction roots

The focused roots are `cryptolab_haetae_mode2_keypair_internal`, `cryptolab_haetae_mode2_signature_i` and `cryptolab_haetae_mode2_verify_internal`. Their source hashes are, respectively, `8b2ddac28621...b78` `fd58271f3ada...ac31`, and `789f36e71ab7...8b06`. Fresh extraction produced target hashes `70716a5cf03d...4565`, `5f632a19c7db...f02f`, and `b745f8919f2c...142b`. The exact hashes and commands are operationally pinned in `manifests/generated-extractions.sha256` and `manifests/focused-extraction.md`.

The closure is intentionally caller-rooted: KeyGen contains the two external exporters, Sign contains the SK importer and `_sf_mu_rawpre`, and Verify contains the raw wrapper, internal verifier, `_sign_verify_tail_m23`, and `__verify_hash_mu`. `RawApiMuReachability.raw_sign_hash_extract` and `raw_verify_hash_extraction_identity` prove exact pRHL equivalence between the raw-ABI and earlier focused hash extractions. Procedure names and source provenance alone are therefore not used as an interchange rule.

11.2 The int/word boundary

`OBL-API-ADDRESS-BINDING` uses the definitions

$$\mathsf{canonical\_ui64\_address}(a) := 0 \le a < 2^{64}, \qquad \mathsf{canonical\_region}(a,n) := 0 \le a \wedge 0 \le n \wedge a+n \le 2^{64}.$$

The central compiled statement is

$$\mathsf{canonical\_ui64\_address}(a) \Longrightarrow \mathrm{to\_uint}(\mathrm{of\_int}(a)) = a.$$

Theorems `canonical_region_valid_region_int` and `canonical_region_valid_region_w64` transport one region contract to the two generated calling conventions. The mode-2 specializations fix $n \in \{992, 1408, 1474\}$. All premises are explicit: modular reduction is not silently ignored outside the canonical range. The claim is PROVED, while actual C/Jasmin allocation and memory safety remain runtime contracts.

11.3 Non-vacuity

The earlier adjacent-region witness provides disjoint valid VK and SK ranges, the new address bridge gives concrete canonical instances, and `stable_region_refl` proves that the stability contract has a model. These witnesses show satisfiability only; they do not replace actual caller reachability.

12 Week 4: caller traces and memory timeline

12.1 Data-flow target

The intended composition is

$$(vk, sk) \longrightarrow \mathsf{export}(vku, sku) \longrightarrow \mathsf{import\_sk}(sku) \longrightarrow \mathsf{hash}_{\mathrm{sign}} \\ \equiv \mathsf{hash}_{\mathrm{verify}} \longleftarrow \mathsf{raw\_vk}(vku).$$

At the already proved local hash boundary this would yield

$$\mathsf{take}_{32}(\mu^{64}_{\mathrm{sign}}) = \mu^{32}_{\mathrm{verify}}.$$

12.2 KeyGen

`keypair_raw_api_exact_export_trace` places the actual `cryptolab_haetae_mode2_keypair_internal` on the left of an exact equivalence and preserves its final memory. The isolated generated exporter theorems prove a 992-byte VK write, a 1408-byte SK write, and preservation of the earlier VK range under the displayed VK/SK disjointness premise. `keypair_raw_api_exports_matching_prefixes_` combines those postconditions with the returned-array prefix theorem.

The attempted single caller Hoare theorem reached the final post-export continuation but its whole-array existential postcondition caused unbounded normalization under both `auto` and `wp`. A constructed-memory replacement was rejected because it would evade the research question. The exact remaining edge is `keypair_raw_api_exports_matching_prefixes_residual`; therefore the KeyGen caller claim is PARTIAL.

12.3 Sign

`sign_raw_api_exact_mu_trace` directly relates the actual raw Sign caller to a trace mirror and preserves final memory and return value. The mirror performs the actual signature-core continuation after recording the imported SK and mu arguments. The compiled companion theorems prove import length 1408, usable prefix length 992, and exact propagation of raw API pre/message addresses and lengths. This is stronger than a standalone observer that stops after hashing.

12.4 Verify and the early-return boundary

Exact trace equivalences compile for `_sign_verify_tail_m23`, `_verify_full_m23`, `_verify_full_mode2`, and `sign_verify_internal_mode2_jazz`. However, the public/raw caller may reject before the tail: signature unpacking, norm rejection, and the final mismatch test all precede the mu call. Hence *siglen* = 1474 is necessary but insufficient. The missing lifts are recorded as `verify_raw_api_exact_mu_trace_residual_obligation` and `cryptolab_verify_internal_exact_mu_trac` an unconditional actual-caller hash-reach theorem would overclaim.

12.5 Ownership and aliasing

Regions	Classification	Reason
vku, sku	MUST-DISJOINT	SK export must frame the prior VK export.
$seedu$, outputs	MAY-ALIAS-BECAUSE-COPIED-FIRST	Seed is fully imported before output writes.
$sku, sigu$	MAY-ALIAS locally; MUST-DISJOINT for reuse	Sign imports first, but a later Verify experiment needs stable keys.
$vku, sigu/siglenu$	MUST-DISJOINT for composition	Sign output must preserve the VK later read by Verify.
$preu, message$	READ-ONLY-ALIAS-ALLOWED	Both are hash inputs; lengths still define distinct byte streams.
$rndu, sigu$	MAY-ALIAS-BECAUSE-COPIED-FIRST	Randomness is copied before signature export.
$sigu, siglenu$	MUST-DISJOINT	The length write must not corrupt the signature.

Cross-call stability is an experiment contract, not a cryptographic assumption. EasyCrypt's total memory map is not treated as a memory-safety proof.

13 Week 4 result and next gate

13.1 Stores-free theorem

`raw_api_key_memory_reaches_mu_zero_loss` has the exact premises

$$\begin{aligned}&\mathsf{canonical\_ui64\_address}(vku),\\&\mathsf{imported\_prefix}(sk_{\mathrm{local}}, mem, sku, 992),\\&\forall i \in [0, 992).\ mem[sku + i] = mem[vku + i],\end{aligned}$$

and concludes both the int/word address equality and

$$\mathsf{sk\_memory\_prefix}(sk_{\mathrm{local}}, mem, \mathrm{of\_int}(vku)).$$

The theorem contains no `stores mem 0` construction. Together with the Week 3 generated theorem, these premises are sufficient for

$$\mathsf{take}_{32}(\mu^{64}_{\mathrm{sign}}) = \mu^{32}_{\mathrm{verify}}$$

and then equal position-64 challenge suffixes. They have not yet been shown to arise from the complete actual caller chain.

13.2 Claim decision

The operational ledger records:

`OBL-API-ADDRESS-BINDING`	PROVED,
`OBL-API-KEYGEN-EXPORT-CALLER`	PARTIAL,
`OBL-API-SIGN-MU-REACHABILITY`	PROVED (exact trace),
`OBL-API-VERIFY-MU-REACHABILITY`	PARTIAL,
`OBL-API-KEY-MEMORY-RAW`	PARTIAL.

It would therefore be unsound to record $\delta_{\mu,\text{raw API}} = 0$. The valid statement remains $\delta_{\mu,\text{raw top}} = 0$, below caller reachability. No full Sign, Verify, encoding, or EUF–CMA delta is inferred.

13.3 Verification and failed decompositions

Every Week 4 theory is a manifested fresh `-no-eco` target. The integrated verifier regenerates all three raw caller targets, checks their hashes, scans proof holes and axioms, rejects a constructed Week 4 stores witness, checks read-only source drift, runs selected baselines, and builds these notes without undefined references or citations.

Rejected proof decompositions were (i) the large direct KeyGen caller postcondition after repeated `auto/wp` stalls and (ii) an unconditional Verify public lift after direct `sim`, explicit-call, and trace-state attempts exposed the early-return gates. Neither failure was hidden behind an axiom.

13.4 Week 5 gate

Week 5 remains on actual caller reachability. The minimal next obligations are a smaller KeyGen export postcondition boundary and a Verify tail-reach trace/predicate tied to Sign-produced canonical signatures. Highbits/LSB and the full challenge call remain out of scope until both edges compile. A second failure should trigger redesign of the unified extraction or trace instrumentation, not another helper-only wrapper.

14 Week 5: sequential KeyGen export

Week 5 closes the actual KeyGen sequential-export boundary. The central compiled theorem is `RawApiKeygenSequentialExport.keypair_raw_api_exports_matching_prefixes`, which opens the real `cryptolab_haetae_mode2_keypair_internal` caller under canonical region and disjoint-output premises and closes with the exact external-prefix relation:

$$\mathsf{res} = \mathsf{W64.zero} \ \wedge \ \mathsf{external\_key\_prefix\_match}(\mathsf{Glob.mem}, sku_0, vku_0).$$

The proof composes three facts:

- the actual 992-byte VK exporter trace;
- the actual 1408-byte SK exporter trace that frames the earlier VK export; and
- the actual KeyGen caller trace with the returned-array prefix theorem.

This is the first week where the sequential export boundary is proved for the real caller rather than a helper-only witness. The theorem does not claim termination, and it does not interpret the raw ABI as a memory-safety theorem.

A strengthened one-line `proc; sim` attempt could not infer the final memory-equality set across the two ghost observation assignments. The compiled replacement splits the common prefix from the exporter tail with `seq 18 20` and aligns both actual copy calls under explicit `Glob.mem`/argument/result equality contracts. Thus the intermediate post-memory is carried through the real second export.

The implication for the week-5 claim ledger is narrow and explicit. The claim

`OBL-API-KEY-MEMORY-RAW-ACCEPT`

remains partial because the accepted Verify path still needs a direct caller-trace bridge, but the KeyGen export side of that bridge is now compiled.

The pinned Jasmin source is:

`haetae-ref-jasmin/jasmin/keypair.jazz`

Its SHA-256 is:

8b2ddac2862122d1c9d58767cbc5caa2caad39c8f41fa8096d66ffa77b783e01

The theorem is checked by the manifest-driven -no-eco compilation in scripts/verify-all.sh; regeneration uses the separate script scripts/extract-raw-api-callers.sh.

15 Week 5: Verify accept tail and input binding

This week makes the accepted Verify boundary explicit instead of assuming it. The compiled trace lemmas show the early-reject tree:

```
siglen <> 1474
  -> immediate reject
siglen = 1474
  -> unpack bad flag -> reject before tail
  -> norm rejection  -> reject before tail
  -> tail_reached and hash-input binding
     -> challenge mismatch -> reject after tail
     -> challenge match    -> accept
```

The accepted-path trace lemmas are:

- verify_full_m23_trace_accept_implies_tail_reached;
- verify_raw_api_trace_accept_implies_tail_reached;
- verify_cryptolab_trace_accept_implies_tail_reached.

The corresponding binding lemmas are:

- verify_tail_trace_binds_hash_inputs;
- verify_full_m23_trace_accept_binds_hash_inputs;
- verify_full_mode2_trace_accept_binds_hash_inputs;
- verify_internal_mode2_trace_accept_binds_hash_inputs;
- verify_raw_api_trace_accept_binds_hash_inputs;
- verify_cryptolab_trace_accept_binds_hash_inputs.

The actual caller lift is actual_raw_verify_accept_binds_hash_inputs. It proves that an accepted raw Verify execution reaches the tail and binds the descriptor inputs exactly:

$$vku = vku_0, \quad preu = preu_0, \quad prelen = prelen_0,$$
$$mu = mu_0, \quad mlen = mlen_0, \quad siglen = 1474.$$

This is the boundary that separates accepted-path reasoning from public Verify correctness. It proves reachability and input binding, not that every legitimate Sign output is accepted.

OBL-SIGN-OUTPUT-TAIL-REACH is therefore the next long-term gate: the report now has the exact early-reject tree, but it still needs a theorem that a legitimate Sign output survives the unpacking and norm gates. Final challenge matching belongs to the separate full Sign/Verify correctness obligation.

The pinned Verify source is:

haetae-ref-jasmin/jasmin/verify.jazz

Its SHA-256 and the raw generated-target SHA-256 are, respectively:

789f36e71ab7784cc37de50ceb04f8d59276f3d1a721acdc64a0be0299f18b06
b745f8919f2ce8c2ff80e63a2330da6cdcc6cad414f341fe90081b886825142b

Fresh checking uses manifests/proof-targets.txt through scripts/verify-all.sh.

16 Week 5: accepted API composition

The accepted-path composition closes two more adapters without claiming the full caller theorem.

16.1 Accepted hash-call adapter

`RawApiAcceptedMuComposition.raw_api_accepting_execution_hash_mu_zero_loss` is the compiled accepted-path adapter from actual caller observations to the generated hash-call theorem. It reuses:

- the actual Sign trace observations;
- the actual accepted Verify trace observations;
- the imported-prefix and external-key-prefix premises; and
- the canonical raw/pre-message region premises.

Its conclusion is

$$\mathsf{mu32\_prefix}(\mathsf{res}_1, \mathsf{res}_2),$$

where the two results are the actual generated Sign and Verify hash-call procedures. The theorem does *not* claim equality of the two trace modules' stored `observed_mu` fields. That direct equality remains open, and the report therefore does not record $\delta_{\mu,\mathrm{raw\_api\_accept}} = 0$.

16.2 Generated helper suffix adapter

In theory `RawApiAcceptedMuComposition`, lemma

```
raw_api_accepting_execution_generated_challenge_suffix_zero_loss
```

extends the accepted-path adapter to the generated Sign/Verify challenge suffix helpers. The theorem is still below the public caller boundary: it requires the actual accepted-path trace premises plus `challenge_pos = 64`, and it concludes exact equality of the generated helper results.

16.3 Diagram and boundary

Stage	Status	Boundary note
Actual KeyGen sequential export	PROVED	Sequential export is closed for the real caller.
Actual Sign caller trace	PROVED	Exact trace exists, but the sign-output frame/stability theorem is missing.
Accepted Verify tail/input binding	PROVED	The accepted path reaches the tail and exposes the exact descriptor inputs.
Accepted hash-call adapter	PROVED	Actual caller observations instantiate the generated hash-call theorem.
Generated helper suffix adapter	PROVED	Accepted trace premises instantiate the position-64 helper composition.
Direct actual-caller mu equality	OPEN	The trace modules are not yet connected by a direct stored-mu equality.

```
Actual Sign trace + actual accepted Verify trace
  -> accepted hash-call adapter PROVED
  -> generated helper suffix adapter PROVED
  -> direct actual-caller stored-mu equality OPEN
```

The implication for the claim ledger is clear: `OBL-API-KEY-MEMORY-RAW-ACCEPT` remains partial, and `OBL-SIGN-VERIFY-CORRECTNESS` stays blocked until the Sign-output tail-reach and correctness obligations are discharged.

The theorem boundary also explains the remaining gap: the accepted-path composition does not synthesize a `stores` witness, and it does not reach the full public Sign/Verify challenge call. It only closes the accepted-path adapter and the helper-level suffix composition.

The Sign and Verify Jasmin sources are pinned respectively at SHA-256:

`fd58271f3ada888dfcef4bcf89027c986cc91464a7cb8446b6de11e40037ac31`
`789f36e71ab7784cc37de50ceb04f8d59276f3d1a721acdc64a0be0299f18b06`

Both the accepted hash adapter and helper suffix adapter are fresh-checked by `scripts/verify-all.sh`. No theorem in this section uses an authored SHAKE axiom or a constructed `stores mem 0` witness.

17 Week 6: actual Sign output frame

17.1 Research question and source boundary

The question for `OBL-SIGN-OUTPUT-FRAME` is whether the actual raw mode-2 Sign ABI can publish its signature and length without changing the external VK, SK, pre, or message bytes that a later Verify call reads. The paper's signing algorithm (Figure 8 in the pinned official PDF) specifies the signature value, while buffer publication is an implementation refinement step. The pinned Sign source and generated raw target hashes are:
`sign.jazz`:

`fd58271f3ada888dfcef4bcf89027c986cc91464a7cb8446b6de11e40037ac31`

`RawSignApiTarget.ec`:

`5f632a19c7db616ca75f4f2f0b319b1a8ba821f0bd552613f0c77290b19ef02f`

17.2 Compiled theorem and premises

Theory `RawApiSignOutputFrame.ec` opens the generated `_api_copy_2948_to_raw`. Lemma `sign_output_copy_exact_and_frames_reused` proves exact publication of 1474 bytes and a byte frame outside the destination interval. It is then composed with the actual 8-byte `siglenu` store.

The actual caller theorem is `sign_raw_api_frames_reused_regions`. Its premises are:

- the initial memory and concrete raw ABI arguments;
- a valid, non-wrapping 1474-byte signature output region;
- disjointness of signature and siglen writes from VK(992), SK(1408), pre, and message regions; and
- disjointness of the signature and siglen output regions.

For every terminating execution it concludes success, stability of all four reused regions, and the value 1474 at `siglenu`. This is Hoare partial correctness: it does not prove termination of the signing rejection loop.

17.3 Invariant, transfer, and non-vacuity

The copy-loop invariant records exact bytes already written and a frame for all addresses outside the full output interval. The subsequent `W64` store is handled bytewise and composed by transitivity. Exact equivalence `sign_raw_api_exact_mu_trace` transfers the trace result to the actual `cryptolab_haetae_mode2_signature_internal` procedure while preserving result and final memory.

The earlier attempt to name the procedure-local `sigp` array directly after a sequence cut failed because that local was outside the theorem instantiation scope. The replacement is a source-independent frame lemma for the generated copy; exact copied bytes remain in the separate copy theorem. The contract is non-vacuous by the following concrete-witness lemma:

`sign_output_frame_contract_satisfiable`.

It supplies pairwise-separated regions.

Status: PROVED as partial correctness.

18 Week 6: region-local mu equivalence

18.1 Why whole-memory equality is wrong

The Sign hash runs before signature publication, whereas the Verify hash runs after the 1474-byte signature and 8-byte length writes. Thus the two global memories are generally unequal even when every hash input byte is preserved. The correct relation is

$$\mathsf{region\_eq}(M_1, M_2, b, n) := \forall i \in [0, n),\ M_1[b+i] = M_2[b+i].$$

The Week 6 data flow is:

$$\begin{aligned} M_{KG} &\xrightarrow{\text{SK import}} \mathsf{SignMu}^{64} \\ \searrow\ &\text{signature publication} \rightarrow M_{after\,Sign} \\ &\text{preserved VK/pre/message} \rightarrow \mathsf{VerifyMu}^{32}. \end{aligned}$$

18.2 Generated procedure theorem

Theory `RegionLocalMuEquivalence.ec` proves locality of the actual generated pre/message absorb loops, including the load-index and W64 tail invariants. Key absorption is separate: Sign reads the local packed SK array, while Verify reads the external VK bytes.

The final theorem

`RegionLocalMuTop.sign_verify_generated_raw_mu_prefix_regionwise`

has the actual generated procedures `_sf_mu_rawpre` and `__verify_hash_mu` on its two sides. Its premises expose mode-2 lengths, `raw_prelen`, non-wrapping pre/message regions, the local-SK/external-VK 992-byte prefix, and byte equality only on the two address ranges read by the hash. Its conclusion is

$$\mathsf{take}_{32}(\mu^{64}_{\mathrm{Sign}}) = \mu^{32}_{\mathrm{Verify}}.$$

No SHAKE prefix or "outside writes do not matter" axiom is used. Init, Keccak permutation, finalize, and both squeeze procedures are the pinned generated programs. The generated focused hashes remain `SignMuHashTarget.ec`:

`c2232e64adf71ee54763f51e11862938c8caa5ca278af30e4c83756091e83aa7`

`VerifyMuHashTarget.ec`:

`fcee969af0b567447c6c9b9c53e29245375ec2bfd54ae11afecdf0a6ebc0ebb7`

18.3 Address subtlety

The predicate `canonical_region(a,n)` allows the empty boundary case $a = 2^{64}, n = 0$. It therefore cannot by itself justify to_uint(of_int(a)) = a. The caller bridge keeps `canonical_ui64_address` premises for each address or length whose exact round trip is used. This prevents a silent modular-wrap premise.

Status: PROVED for mode-2 raw/internal hashing.

19 Week 6: direct API trace boundary

19.1 Actual sequential trace

`RawApiDirectObservedMu.ec` preserves the real execution order:

```
actual Sign raw ABI -> publish signature -> actual Verify raw ABI
       |                                          |
       v                                          v
SignRawApiMuTrace.run -> VerifyCryptolabMuTrace.run
```

The same `sigu`, pre, and message arguments are used by both calls. The exact theorem is

`raw_sign_then_verify_actual_exact_trace`.

It proves equal result pairs and equal final memory. The traces add observations but retain the entire signature core, verification residual computation, and rejection behavior.

On the trace sequence, `sign_then_verify_trace_accept_implies_tail_reached` proves

$$\mathsf{VerifyAccept} \Longrightarrow \mathsf{TailRan},$$

The companion binding lemma

`sign_then_verify_accept_binds_observed_inputs`

derives both sets of VK/pre/message inputs from the post-state. No `tail_reached` or observed hash value occurs in either precondition.

The intended full implication is

$$\mathsf{VerifyAccept} \Longrightarrow \mathsf{TailRan} \Longrightarrow \mathsf{take}_{32}(\mathsf{SignTrace.observed\_mu}) = \mathsf{VerifyTrace.observed\_mu}.$$

19.2 What remains open

The last implication is not yet compiled. The region-local theorem is a pRHL relation between two hash procedure executions, while the desired result is a unary postcondition about values stored by two calls that have already completed inside one sequential trace. A product program, deterministic functional hash specification, or justified replay rule is needed to transport the former into the latter.

Repeating the Week 5 construction by placing both observations in a hash-call precondition was rejected: it does not establish reachability from trace execution. Adding an independent replay without proving equality to the original observations was also rejected.

The remaining semantic step is named

`OBL-MU-PRODUCT-REPLAY`.

The usual `byequiv` pattern does not directly convert the compiled pRHL judgment into the required unary Hoare postcondition. Aligning the two full traces relationally would leave the post-hash Sign rejection continuation unpaired; eliminating it one-sidedly would require the still-open signing-loop losslessness proof. That termination premise is outside this partial-correctness seam and was not hidden.

Therefore:

- `OBL-DIRECT-OBSERVED-MU`: PARTIAL;
- `OBL-API-KEY-MEMORY-RAW-ACCEPT`: PARTIAL;
- $\delta_{\mu,\text{raw\_api\_accept}} = 0$: not claimed.

There is also no accepted Sign-output witness. That separate fact belongs to the functional obligation

`OBL-SIGN-OUTPUT-TAIL-REACH`,

not to the accepted-forgery adapter.

20 Six-week checkpoint

The operational status is summarized in `SIX_WEEK_REVIEW.md`; this section is the narrative record rather than a second claim ledger.

Lane	Status	Evidence boundary
KeyGen VK/SK prefix/export	PARTIAL FC	Actual terminating raw caller export is proved; retries and distribution are open.
Sign output frame	PROVED	Actual raw caller frames disjoint VK/SK/pre/message regions.
Accepted Verify control	PROVED	Actual success implies tail execution and exact hash input binding.
Generated raw mu relation	PROVED	Actual hash calls are related with region-local memories.
Direct stored trace mu	PARTIAL	Exact sequential trace exists; product/replay transport is missing.
Legitimate Sign correctness	BLOCKED	Pack/unpack, norm, hint, response, and arithmetic remain.
Distribution/security	BLOCKED	Rejection, entropy, forking, and final advantage composition remain.

The smallest defensible paper result is a machine-checked collection of mode-2 raw/internal partial-correctness and accepted-event adapters with explicit memory premises. It is not a full functional-correctness theorem and not a public-API EUF-CMA theorem.

The next gate is deliberately bounded. One sprint may test an explicit product/replay construction. If it does not compile, the accepted generated hash-call adapter must be frozen as the stated paper boundary, and the main effort should move to pack/unpack and norm correctness or another independent security lane. More proof-only caller wrappers would not change the semantic gap.

21 Week 7: Product/Replay Feasibility Gate

Week 7 treated `OBL-MU-PRODUCT-REPLAY` as a bounded feasibility gate, not as a lane to expand indefinitely. The success criterion was strict: the theorem surface had to contain the actual generated procedures `SignMuHashTarget.M._sf_mu_rawpre` and `VerifyMuHashTarget.M.__verify_hash_mu`, and the post-state values stored in the actual trace modules had to be related directly.

The desired surface remained:

$$\mathsf{VerifyAccept} \Longrightarrow \mu 32\_prefix(\mu^{64}_{\text{sign,obs}}, \mu^{32}_{\text{verify,obs}}).$$

The two symbols denote these post-state fields:

- Sign observation: `SignRawApiMuTrace.observed_mu`;
- Verify observation: `VerifyCryptolabMuTrace.observed_mu`.

The compiled prerequisites from Weeks 5–6 remain valid:

- actual generated returned-value hash equality on the raw branch,
- exact actual-to-trace preservation for the sequential Sign→Verify trace, and
- accepted Verify control and actual hash-input binding.

The missing step is a semantic transport:

$$\text{return-value pRHL theorem} \Longrightarrow \text{stored post-state observation equality.}$$

The obstruction is not another memory-frame lemma. It is the gap between two already completed trace executions and the returned values of the actual generated hash calls embedded inside their residual continuations. The Sign and Verify continuations are different, and Week 7 deliberately refused four unsound escapes:

- adding another caller/observation wrapper,
- assuming observation equality as a precondition,
- adding a SHAKE locality axiom, or
- discarding the unmatched Sign continuation without the still-open losslessness obligations.

Concretely, the compiled pRHL goal names `res{1}` and `res{2}` returned by the two generated hash procedures. The unary sequential post-state names only the two module fields `observed_mu`. After both trace calls return, the local hash-return variables are no longer in scope. Applying the pRHL theorem earlier leaves the unequal residual continuations; one-sided `seq` elimination requires the unproved Sign losslessness result. Thus the failed prototype is a program logic surface mismatch, not an SMT arithmetic failure. Direct `sim`, `seq`, and consequence were not retried through another wrapper.

Accordingly, the Week 7 decision is:

- `OBL-MU-PRODUCT-REPLAY`: **DEFERRED**,
- paper boundary: **explicit**,
- `OBL-DIRECT-OBSERVED-MU`: remains **PARTIAL**,
- no claim of $\delta_{\mu,\text{raw\_api\_accept}} = 0$.

22 Week 7: Mode-2 Signature Codec

The main Week 7 lane moved to mode-2 signature serialization. The actual standalone wrappers `pack_sig_mode2_full_jazz` and `unpack_sig_mode2_full_jazz` were regenerated and pinned by hash. Their reachable closures contain the actual prefix, full, and rANS helper procedures used below.

Audited parameters and 1474-byte layout

The extracted wrapper calls agree on:

$$\begin{aligned}&\mathsf{sigbytes} = 1474,\ \mathsf{lcount} = 4,\ \mathsf{hb\_count} = 1024,\ \mathsf{hb\_m} = 13,\ \mathsf{hb\_offset} = 6,\\&\mathsf{h\_count} = 512,\ \mathsf{h\_m} = 13,\ \mathsf{h\_offset} = 239,\\&\mathsf{base\_hb} = 132,\ \mathsf{base\_h} = 7,\ \mathsf{payload\_limit} = 416.\end{aligned}$$

The byte structure recovered from the actual extracted code is:

$[0, 32)$	256 challenge bits, packed within 32 bytes
$[32, 1056)$	1024 signed low-coefficient bytes
$[1056, 1058)$	hb/h encoded-size deltas
$[1058, 1058 + \mathsf{hbsize})$	hbz encoded payload
following bytes	h encoded payload
through byte 1473	canonical zero padding

The arithmetic boundary is $1056 + 2 + 416 = 1474$. The unpacker additionally checks nonzero size values, the two base-offset ranges, their total against 416, and zero trailing bytes.

Actual prefix procedures

The authored theory first proves two procedure-level layout theorems. The packer theorem opens `_pack_sig_prefix` and establishes the challenge-bit and 1024-byte low layouts. The unpacker theorem opens `_unpack_sig_prefix` and establishes the inverse bit extraction and signed-byte extension. Its postcondition also preserves low-array words $[1024, 2048)$.

The sequential harness calls the two actual generated procedures:

```
sig <@ Pack._pack_sig_prefix(sig, cp, low, W64.of_int 4,
                             W64.of_int 1474);
(decoded_cp, decoded_low) <@
  Unpack._unpack_sig_prefix(decoded_cp, decoded_low, sig,
                            W64.of_int 4);
```

The fresh-compiled theorem is `pack_unpack_sig_prefix_mode2_roundtrip`. Its exact semantic statement is:

$$\begin{aligned}&\mathsf{canonical\_challenge}(cp)\ \wedge\ \mathsf{canonical\_signed\_low}(low)\\&\quad\Longrightarrow \mathsf{challenge\_prefix\_eq}(decoded\_cp, cp)\ \wedge\\&\qquad \mathsf{low\_mode2\_eq}(decoded\_low, low)\ \wedge\ \mathsf{low\_tail\_frame}(initial\_low, decoded\_low).\end{aligned}$$

Here challenge words are exactly zero or one. A canonical low word has signed value in $[-128, 128)$ and equals the arithmetic sign extension of its low byte. The theorem is Hoare partial correctness; it does not assert signing loop termination. The compiled `prefix_codec_preconditions_satisfiable` lemma supplies all-zero arrays, establishing non-vacuity.

Proof strategy and failed decomposition

The pack proof uses three invariants: zero-fill progress, bit-by-bit assembly of each challenge byte, and the low-byte store cursor. The unpack proof uses a nested byte/bit invariant for challenge recovery and a signed-extension invariant for low coefficients. The initial composition attempt fixed the signature to an external logical parameter. It failed because the local program variable returned by the preceding pack call could not instantiate that pure parameter in a `call` tactic. Generalizing the unpack theorem to predicates over its current `sigp` argument closed the composition; repeating the fixed-array or abstract-function approach should be avoided.

An attempted monolithic pack tail-frame proof was not retained. The compiled boundary proves the unpack low-array tail, while the pack signature-array $[1474, 2948)$ frame and challenge-output tail remain `OBL-SIG-PREFIX-FRAME`. The standalone codec procedures do not write global memory.

Suffix obligations and security boundary

The actual metadata and zero-padding gates are audited but do not yet have procedure theorems. Both rANS pairs remain open:

- `_encode_hb_z1_full` with `_decode_hb_z1_full`;
- `_encode_h_full` with `_decode_h_full`.

The prefix result is security-relevant. Deterministic canonical parsing removes one source of signature ambiguity and malleability. It supplies one input to the adapter between implementation acceptance and the paper forgery event. It does not make the full encoding loss zero: suffix ambiguity, malformed rejection, norm, hint, arithmetic, and challenge consistency remain open. Consequently neither Sign-output tail reach nor full Verify correctness follows from this chapter.

23 Week 7 Result and Next Gate

Week 7 ends with a deliberate split:

- the accepted-path caller-plumbing lane is frozen at an explicit paper boundary before direct stored-μ equality;
- the main forward lane is the mode-2 signature codec.

The practical Week 8 gate is therefore simple.

GO-B—reached. The actual prefix codec inverse compiles. Week 8 therefore starts with the single obligation `OBL-SIG-HBZ-ENCODE-DECODE` for the actual `_encode_hb_z1_full`/`_decode_hb_z1_full` pair. The hint codec and canonical suffix composition follow only after that invariant is established.

The key outcome of Week 7 is not a new zero-loss security claim. It is a cleaner proof frontier:

FREEZE-A at replay boundary and GO-B to the actual suffix codec.

The result does not prove full pack/unpack, a zero encoding delta, norm-gate success, Sign termination, or legitimate-signature acceptance. Those remain visible nodes in the theorem graph.

24 Week 8: Mode-2 HBZ and rANS Boundary

Week 8 keeps the Week 7 product/replay freeze unchanged and moves only along `OBL-SIG-HBZ-ENCODE-DECODE`. The result is a compiled leaf inverse, an exact focused/full extraction identity for the actual HBZ wrappers, a compiled generic mode-2 arithmetic certificate, and a compiled concrete actual-array table corollary. The generated encoder/decoder loop refinement remains the gates to a full actual HBZ round trip, so the parent claim remains PARTIAL.

Actual extraction and implementation graph

The focused roots are `encode_hb_z1_prepare_jazz`, `decode_hb_z1_apply_jazz`, `rans_encode_jazz`, `rans_decode_jazz`, and the two mode-2 full wrappers. Their generated closures form

`encode_hb_z1_mode2_full_jazz` → `_encode_hb_z1_full` → {`_encode_hb_z1_prepare`, `_rans_encode`, `__c`

`decode_hb_z1_mode2_full_jazz` → `_decode_hb_z1_full` → {`_rans_decode`, `_decode_hb_z1`

The wrappers fix $\mathsf{count} = 1024$, $m = 13$, and $\mathsf{offset} = 6$. Regeneration checks show byte-identical generated bodies for these procedures in the focused closure and in the Week 7 actual signature pack/unpack closures. The pinned source hashes include `dff14e57...3516cd` for `encoding.jinc`, `818bb254...127b49` for `sparse_encoding.jinc`, and `b1d4b2dd...228429a` for `encoding_tables.jinc`.

Coefficient-to-symbol mapping

For the first 1024 words, canonical mode-2 input means

$$-6 \le \mathsf{signed}(hbz[i]) < 7.$$

The actual prepare loop establishes

$$\mathsf{symbol}[i] = \mathsf{signed}(hbz[i]) + 6, \qquad 0 \le \mathsf{symbol}[i] < 13,$$

sets its bad word to zero, and preserves symbol bytes $[1024, 2048)$. The actual apply loop zero-extends the symbol and subtracts six modulo 2^{32}. Its word-level inverse lemma handles negative input coefficients and preserves coefficient words $[1024, 2048)$.

The sequential actual-procedure theorem is `encode_prepare_decode_apply_mode2_inverse`. Its sole semantic input is `canonical_hbz_mode2`; the conclusion contains prepare success, exact first-1024 recovery, and both tail frames. It is Hoare partial correctness, not a termination claim. The all-zero array supplies a compiled precondition witness.

Concrete 13-symbol table certificate

The concrete mode-2 intervals are:

s	0	1	2	3	4	5	6	7	8	9	10	11	12
c_s	0	1	2	3	8	66	312	710	957	1016	1021	1022	1023
f_s	1	1	1	5	58	246	398	247	59	5	1	1	1

Every frequency is positive and the sum is 1024. The adjacent intervals partition $[0, 1024)$. The authored certificate checks the generic encoder field formulas, decoder packed start/frequency formulas, and the reciprocal division arithmetic used by the mode-2 design. The concrete actual-array corollary is intentionally split out into `Mode2HbzSymbolWordsGenerated`; the external generator is only a deterministic producer of EasyCrypt proof text, and the actual-array link is accepted only because that generated theory itself fresh-compiles.

For normalized state x, the generic certificate also proves exactness of the reciprocal multiply/high-half/shift computation. Frequency-one symbols use the implementation's $x - 1$ quotient and compensated bias. In every case the actual fast expression equals

$$E_s(x) = \left\lfloor \frac{x}{f_s} \right\rfloor 1024 + c_s + (x \bmod f_s),$$

with a non-overflowing W64 product and a W32 state below 2^{31}.

State machine, normalization, and inverse invariant

The state lower bound is $L = 2^{23}$ and the scale is 2^{10}. Encoding traverses symbols backward. Before E_s, bytes are emitted from the low end of x while $x \geq 2^{21} f_s$, using a cursor that moves backward. The final state is written as four little-endian bytes. Decoding reads those bytes, recovers symbols forward from the low 10-bit slot, applies

$$D_s(x') = \left\lfloor \frac{x'}{1024} \right\rfloor f_s + (x' \bmod 1024) - c_s,$$

and consumes normalization bytes while the state is below L.

The pure step theorem is now compiled. In `Mode2RansCore`, `pure_rans_step_inverse` proves $D_s(E_s(x)) = x$, and `hbz_fast_step_decode_inverse` replaces E_s with the concrete reciprocal implementation formula under $0 \leq s < 13$ and $1 \leq x < \mathsf{hbz\_xmax}(s)$. The remaining generated-loop invariant must additionally relate the encoder's backward byte stack $[off, count)$ to the decoder's forward input, preserve $L \leq x < 2^{31}$ at symbol boundaries, and track failure before the four-byte state area. A prototype for the actual suffix copy discharged its body invariant but did not finish the quantified loop-exit projection under fresh compilation; no such theorem is retained as compiled evidence.

Success-conditioned boundary and failure branch

Canonical coefficient bounds do not imply buffer success. If the backward encoder exhausts its fixed workspace, the actual full encoder returns $\mathsf{size} = 0$. The target theorem must therefore have the form

$$\mathsf{size} = 0 \quad \vee \quad (4 \leq \mathsf{size} \leq 1024 \ \wedge \ \mathsf{decode\_bad} = 0 \ \wedge \ \mathit{decoded}[0..1024) = \mathit{original}[0..1024)) \,.$$

Neither decoder success nor output equality may be assumed. A successful actual encoder/decoder witness is also required before the parent can be promoted to PROVED.

The authored proof hashes are `909e4eed...0d865f` (prepare), `03ac5595...521d4` (apply), `7da906bc...92e55` (leaf composition), `8bdecef8...6ad58` (table arithmetic), `3f664576...d6ba90` (generated table certificate), `6efb58b5...5ba80` (actual extraction boundary), and `a4a10f55...a5838c` (pure rANS step). Each is checked by `easycrypt compile -script -no-eco` through `scripts/verify-all.sh`; the extraction and generated certificate are regenerated before those compilations.

Proof status, failed tactic, and security boundary

Obligation	Status
HBZ prepare/apply and leaf frames	PROVED
Generic mode-2 table arithmetic certificate	PROVED
Concrete actual-array table certificate	PROVED
Pure arithmetic rANS step inverse	PROVED
Actual suffix copy and frame	PARTIAL
Actual encoder loop refinement	PARTIAL
Actual decoder loop refinement	BLOCKED
Actual full HBZ left inverse	PARTIAL

A monolithic `inline; sim` attempt is structurally inadequate: it does not invent the backward-stack/forward-consumption relation, and it obscures the size-zero branch. Repeating larger full-procedure inlining is rejected. A weaker actual-encoder control prototype also stalled at its nested normalization loop, after the outer post-loop commands and branch had

been separated with `wp`. The next proof must refine each generated loop against one explicit state/byte-stack model instead of adding another wrapper.

An encode-to-decode left inverse is functional evidence only. Canonical parsing additionally needs malformed rejection or decode-to-re-encode uniqueness. Week 8 therefore proves neither a zero encoding delta nor `OBL-SIGN-OUTPUT-TAIL-REACH`, full Verify correctness, signing losslessness, or implementation EUF-CMA security. Fresh verification uses the aggregate script above and records target-specific logs under `logs/`.

25 Week 9: rANS Byte Stack and Actual Core Boundary

Week 9 has one target, `OBL-RANS-CORE-INVERSE`. The sprint does not reprove the Week 8 coefficient leaves, table certificate, pure step inverse, or full-procedure extraction identity. It fixes a common trace for the reverse encoder and forward decoder, proves the normalization arithmetic and the actual suffix copy, and places the three actual generated calls in one unary harness. The two generated outer-loop refinements remain open, so the result is CONTINUE-RANS, not a completed inverse.

Pinned actual procedures and hashes

The proof uses the Week 8 focused targets without a second extraction:

Generated target	SHA-256 prefix
`RansEncodeTarget.ec`	`3184367f5d41196b`
`RansDecodeTarget.ec`	`2659860ffe3dff9d`
`HbzFullEncodeTarget.ec`	`6c085864d31a3cb4`

The source hashes remain `c2caff5b...e1f4e9` for `hpoly.jazz` and `e800d1d9...b5cbd` for `encoding.jazz`. The generated procedures are:

- `RansEncodeTarget.M._rans_encode`;
- `HbzFullEncodeTarget.M.__copy_encoded_suffix`;
- `RansDecodeTarget.M._rans_decode`.

Extraction regeneration and exact generated hashes are checked before proof compilation.

Reverse/forward state correspondence

For symbols $s_0, \ldots, s_{n-1}$, $n = 1024$, the boundary-state convention is

$$x_n = 2^{23},$$
$$x_0 = \text{the decoder's serialized initial state},$$
$$\text{state after decoding } k \text{ symbols} = x_k.$$

The encoder processes $j = n-1, \ldots, 0$. At step j it normalizes x_{j+1}, applies the fast concrete-table step, and obtains x_j. The decoder processes $k = 0, \ldots, n-1$, selects s_k, applies the inverse arithmetic step, and consumes the byte segment that reconstructs x_{k+1}.

Cursor boundaries use

$$c_0 = 4, \qquad [c_k, c_{k+1}) = \text{normalization bytes for symbol } s_k, \qquad c_n = \mathsf{size}.$$

The first four bytes are $\mathsf{serialize32\_le}(x_0)$. In EasyCrypt these objects are `trace_states`, `trace_segments`, `trace_cuts`, and `trace_bytes`; `valid_rans_trace` merely packages their defining equalities and is not an implementation assumption.

Normalization-byte stack and byte order

For the mode-2 table,

$$2^{23} \le x < 2^{31}, \qquad x_{\max}(s) = 2^{21} f_s \ge 2^{21}.$$

Thus the chosen pure normalization emits zero, one, or two bytes. If $x = 2^{16}q + 256h + \ell$, two encoder iterations write ℓ first and h second while moving the cursor backward. The final decoder-order segment is therefore $[h, \ell]$, and

$$q \xrightarrow{h} 256q + h \xrightarrow{\ell} 2^{16}q + 256h + \ell = x.$$

The compiled leaf results are separated as follows.

- `renorm_bytes_readback` proves the list-level equation;
- `encoder_shift8_uint` and `encoder_low_byte_uint` identify actual W32 shift/truncate with division/remainder; and
- `decoder_append_encoder_byte` proves the one-byte W32 inverse; and
- `encoder_two_byte_decoder_order` proves the two-byte W32 inverse.

The W64 cursor lemmas prove decrement safety under their stated bounds.

The pure state invariant is also compiled:

$$2^{23} \le x_{j+1} < 2^{31} \Longrightarrow 2^{23} \le x_j < 2^{31}.$$

It uses the Week 8 concrete reciprocal-step theorem after proving the reduced state lies in $[1, x_{\max}(s))$.

Encoder and decoder loop invariants

The intended actual encoder outer invariant at counter i is

$$\begin{aligned} 0 \le i \le 1024, \\ x = \mathsf{encode\_trace}(s_i, \ldots, s_{1023}).\mathsf{state}, \\ \mathsf{encp}[off, 1024) = \mathsf{traceBytesWithoutState}(s_i, \ldots, s_{1023}), \\ off + |\mathsf{traceBytesWithoutState}| = 1024. \end{aligned}$$

The inner invariant must additionally relate zero, one, or two actual shifts and backward stores to the corresponding prefix of the pure normalization segment. On a live iteration it must retain enough cursor space for the final four-byte state and frame all unwritten bytes.

The intended actual decoder invariant is

$$\begin{aligned} 0 \le i \le 1024, \\ \mathsf{symsp}[0, i) = (s_0, \ldots, s_{i-1}), \\ x = x_i, \qquad off = c_i, \qquad bad = 0, \\ \mathsf{bufp}[c_i, \mathsf{size}) = \text{the remaining trace suffix.} \end{aligned}$$

At exit it should imply $x = 2^{23}$, $off = \mathsf{size}$, exact symbol recovery, decoder success, and the output-array frame. Neither actual outer invariant is yet compiled.

Actual suffix-copy refinement

The previous quantified-exit residual is closed. The generated-procedure theorem is

`copy_encoded_suffix_correct`.

Its premises are

$$0 \le off_0, \quad 0 \le n, \quad off_0 + n \le 2048$$

and concludes

$$\begin{aligned} \forall j \in [0, n) : &\quad out_{\text{after}}[j] = enc_{\text{before}}[off_0 + j], \\ \forall j \in [n, 2048) : &\quad out_{\text{after}}[j] = out_{\text{before}}[j]. \end{aligned}$$

The proof opens the actual copy loop. It does not construct a desired decoder buffer. Deriving its bounds from the actual encoder-success branch remains in the encoder refinement.

Success/failure disjunction and actual harness

The actual encoder and decoder each have a direct control theorem fixing the concrete mode-2 tables and parameters and proving the published bad word is in $\{0, 1\}$. These are implementation theorems, but their conclusion is control only rather than trace refinement.

The unary `Mode2RansActualHarness.run` executes

$$\texttt{\_rans\_encode} \longrightarrow \begin{cases} bad \neq 0 & : \mathsf{decoder\_ran} = \mathsf{false}, \\ bad = 0 & : \texttt{\_\_copy\_encoded\_suffix}(off, 1024 - off) \\ & \quad \longrightarrow \texttt{\_rans\_decode}(1024, 1024 - off, 13). \end{cases}$$

The theorem `actual_rans_harness_branches_on_encoder_result` proves the branch equivalence and exact decoder input fields. Encoder success is obtained from the actual return value rather than assumed. It does not conclude decoder success or symbol equality.

The desired final disjunction remains

$$\begin{aligned} bad_{enc} \neq 0 \quad \vee \quad & (decoded[0, 1024) = symbols[0, 1024) \\ & \wedge x_{final} = 2^{23} \wedge off_{final} = \mathsf{size} \wedge \mathsf{frames}). \end{aligned}$$

Non-vacuity and exact proof status

The pure symbol/state premises and the decoder-state construction have compiled witnesses. No EasyCrypt theorem yet proves that the all-6 symbol array makes the actual encoder return success with $4 \le \mathsf{size} \le 1024$. Hence `OBL-RANS-ACTUAL-SUCCESS-WITNESS` remains PARTIAL; runtime execution is not used as a theorem.

Obligation	Status	Boundary
Normalization byte inverse	PROVED	pure/word leaf
State-bound preservation	PROVED	pure trace
Actual suffix copy	PROVED	generated procedure
Actual harness control	PROVED	three actual calls, no inverse
Actual encoder refinement	PARTIAL	trace postcondition open
Actual decoder refinement	PARTIAL	trace consumption open
Actual core inverse	PARTIAL	no round-trip conclusion
Actual success witness	PARTIAL	no compiled witness

Failed tactics and next gate

A weak outer-loop invariant can prove $bad \in \{0, 1\}$ and can preserve every early/failure branch. It cannot synthesize the array/list byte-stack relation. Direct encoder/decoder lockstep was rejected because their directions and inner-loop counts differ. A constructed decoder buffer, external `valid_rans_trace` premise, decoder-success premise, 1024-step generated unrolling, and monolithic `inline; sim` are also rejected.

The single Week 10 gate is the actual encoder outer-loop invariant linking $(i, x, off, encp)$ to the suffix of the pure trace and deriving the success-size bounds. Only after it compiles should the actual decoder-consumption invariant be completed. The h codec and security widening remain out of scope.

26 Week 10: Encoder-Only rANS Refinement

Week 10 targets `OBL-RANS-ENCODE-REFINEMENT` and does not modify the actual decoder proof surface. The generated target is `RansEncodeTarget.M._rans_encode`; its regenerated SHA-256 is `3184367f5d41196b...e22cebc5`. The outcome is CONTINUE-ENCODER: actual inner-loop byte semantics and the actual success-size range compile, but the whole returned suffix has not yet been identified with the pure trace.

BArray/list bridge and suffix recurrence

For an actual `BArray2048` input a, the Week 10 view is

$$\mathsf{symbols}(a) = [\mathsf{uint}(a[0]), \ldots, \mathsf{uint}(a[1023])], \qquad \mathsf{suffix}(a, i) = \mathsf{drop}(i, \mathsf{symbols}(a)).$$

The compiled bridge proves

$$|\mathsf{symbols}(a)| = 1024, \quad \mathsf{suffix}(a, 1024) = [], \quad \mathsf{suffix}(a, i) = \mathsf{uint}(a[i]) :: \mathsf{suffix}(a, i + 1).$$

It also defines the pointwise predicates

$$\mathsf{segment\_matches}(A, p, b) \iff \forall k < |b| : \ \mathsf{uint}(A[p + k]) = b[k],$$
$$\mathsf{prefix\_frame}(A_0, A_1, p) \iff \forall k < p : \ A_1[k] = A_0[k].$$

The set/prepend, concatenation, frame, and W64 no-wrap lemmas are all checked by EasyCrypt; the proof does not enumerate 1024 entries.

The pure one-symbol recurrence is

$$\mathsf{encode\_trace}(s :: t).\mathsf{state} = \mathsf{fastStep}(\mathsf{normalize}(\mathsf{encode\_trace}(t).\mathsf{state}, s), s),$$
$$\mathsf{encode\_trace}(s :: t).\mathsf{bytes} = \mathsf{normBytes}(\mathsf{encode\_trace}(t).\mathsf{state}, s) \mathbin{+\!\!+} \mathsf{encode\_trace}(t).\mathsf{bytes}.$$

Concrete word-level update

`Mode2RansEncoderWordStep` models the exact generated update: four table fields at index $4s$, the reciprocal W64 product and high 32 bits, the variable shift ladder, complement multiplication, bias, and W32 addition. The compiled theorem is

$$0 \le s < 13 \ \wedge \ 1 \le \mathsf{uint}(x) < x_{\max}(s) \implies \mathsf{actualWordStep}(x, s) = \mathsf{W32}(\mathsf{fastStep}(\mathsf{uint}(x), s)).$$

The concrete table certificate discharges the lookup, reciprocal quotient, packed-field, overflow, and signed/unsigned obligations. This is a word-level theorem, not yet the outer generated-loop refinement.

Actual inner normalization invariant

The generated normalization body is

$$off \leftarrow off - 1; \qquad encp[off] \leftarrow \mathsf{truncate8}(x); \qquad x \leftarrow x \gg 8.$$

The direct Hoare proof in `Mode2RansEncoderActualInner` over `_rans_encode` installs a phase witness (x_0, off_0, k) satisfying

$$\begin{aligned} 0 \le k \le 4, \qquad \mathsf{uint}(off) &= off_0 - k, \\ \mathsf{uint}(x) &= \mathsf{phaseState}(\mathsf{uint}(x_0), k), \\ \mathsf{segment\_matches}(encp, off_0 - k, &\mathsf{phaseWritten}(\mathsf{uint}(x_0), k)), \\ \mathsf{prefix\_frame}(before, encp, off_0 - k)&. \end{aligned}$$

`encoder_inner_segment_step` proves the actual store prepends the exact low byte in decoder address order and preserves the frame. The W64 decrement is justified before every iteration rather than left to automation.

The bound $k \le 4$ is conservative: it follows from W32 finiteness and positive $x_{\max}$. The pure trace separately proves the sharper mode-2 $k \in \{0, 1, 2\}$. The current actual invariant does not yet bind x_0 to the tail trace, so the two facts cannot yet be identified. Consequently `OBL-RANS-ACTUAL-INNER-NORMALIZE` is PARTIAL, although its actual byte-write/frame step is proved.

Required outer invariant and failure transition

On the live branch the target invariant is

$$\begin{aligned} 0 \le i \le 1024, \\ \mathsf{uint}(x) = \mathsf{encode\_trace}(\mathsf{suffix}(symbols_0, i)).&\\ \mathsf{uint}(off) + |\mathsf{encode\_trace}(\mathsf{suffix}(symbols_0, i)).\mathsf{bytes}| &= 1024, \\ \mathsf{segment\_matches}(encp, \mathsf{uint}(off), \mathsf{encode\_trace}(\mathsf{suffix}(symbols_0, i)).\mathsf{bytes}), \qquad &\mathsf{prefix\_frame}(enc_0, encp, \mathsf{uint}(o \end{aligned}$$

The actual program sets $bad = 1$ when $off < 4$. The failure branch therefore does not retain the live trace equality; subsequent control forces $i = 0$ and skips final-state serialization. Encoder success is never assumed.

The first remaining edge is a tail-aware inner invariant:

$$\mathsf{segment\_matches}\big(encp, off_0 - k, \mathsf{phaseWritten}(x_0, k) +\!\!+ \mathsf{encode\_trace}(tail).\mathsf{bytes}\big).$$

It must show that guard-false implies k equals the pure normalization length; only then can the word-step theorem and suffix recurrence rebuild the outer invariant.

Final state serialization and success size

The four final `set8` operations compile independently as

$$\mathsf{serialize32\_le}(\mathsf{uint}(x)),$$

with exact inside-byte equality and frame outside the four positions. Their intended composition is

$$\mathsf{serialize32\_le}(x) +\!\!+ \mathsf{encode\_trace}(\mathsf{symbols}(a)).\mathsf{bytes} = \mathsf{trace\_bytes}(\mathsf{symbols}(a)).$$

The direct generated-procedure theorem `actual_rans_encode_success_size_bound` preserves the disjunction

$$bad \neq 0 \quad \vee \quad (bad = 0 \wedge 0 \le off \le 1020 \wedge 4 \le 1024 - off \le 1024).$$

This is Hoare partial correctness. It neither proves local encoder losslessness nor supplies an actual success witness.

Status, non-vacuity, and rejected tactics

The array/list, word-step, pure progress, serialization, canonical-symbol, and size premises have compiled witnesses. No theorem proves that the all-6 symbol input terminates with $bad = 0$, so `OBL-RANS-ACTUAL-SUCCESS-WITNESS` remains PARTIAL. Local fixed-count encoder losslessness is distinct from Sign rejection-loop losslessness.

Evidence	Status
BArray/list and suffix recurrence	PROVED
Concrete encoder word step	PROVED (word level)
Actual inner byte/frame transition	PROVED
Exact actual 0/1/2 inner exit	PARTIAL
Final LE serialization	PROVED (leaf)
Actual success size range	PROVED (partial correctness)
Whole actual suffix equals pure trace	PARTIAL
Actual success witness	PARTIAL

A monolithic `inline; sim`, direct encoder/decoder lockstep, an arbitrary replay buffer, success or trace equality as a precondition, and 1024-step concrete unrolling were rejected. The next gate is only the tail-aware actual inner exit and one outer semantic transition. Decoder semantics, full HBZ composition, and wider security claims remain out of scope.

27 Week 11: Actual Encoder Trace Closure

Week 11 closes `OBL-RANS-ENCODE-REFINEMENT` as PROVED, SUCCESS-CONDITIONED PARTIAL CORRECTNESS. The decisive theorem opens the actual generated `RansEncodeTarget.M._rans_encode`; it is not a wrapper or a pure-only encoder theorem. Decoder refinement, the core inverse, and the full HBZ wrapper remain PARTIAL.

Why the Week 10 local invariant was insufficient

The Week 10 predicate remembered an arbitrary array snapshot immediately before one normalization loop. It could prove a local store and frame, but lost the normalization bytes written by earlier outer iterations. The actual encoder needs the continuation-aware relation

$$\mathsf{segment\_matches}(encp, off_0 - k, \mathsf{innerWritten}(x_0, s, k) +\!\!+ \mathsf{encodeTrace}(tail).\mathsf{bytes}).$$

Week 11 therefore replaces the arbitrary snapshot with predicates relative to the initial encoder array and the exact pure tail of the actual symbol array.

Tail-aware inner invariant and exact exit

For outer index j, let

$$tail = \mathsf{symbolSuffix}(symbols_0, j+1), \qquad x_0 = \mathsf{encodeTrace}(tail).\mathsf{state}.$$

The compiled `encoder_inner_tail_inv` carries

$$0 \le k \le total \le 2, \quad x = \mathsf{innerStateAfter}(x_0, k), \quad off = off_0 - k,$$

the concatenated byte segment above, and $\mathsf{prefixFrame}(enc_0, encp, off_0 - k)$. Each actual generated decrement, `set8`, and right shift preserves this predicate. At guard exit, `encoder_inner_tail_exit_exact` uses the actual $x_{\max} \le x$ guard and the mode-2 normalization bound to prove

$$k = \mathsf{mode2NormalizationLen}(x_0, s),$$

so the written list is exactly $\mathsf{mode2NormalizationBytes}(x_0, s)$.

Actual table-load and word-step bridge

The generated weakest-precondition exposes literal loads for $x_{\max}$, reciprocal frequency, bias, packed complement/frequency, and shift. The theorem family in `Mode2RansEncoderGeneratedWordStep` proves those loads, the high-half W64 product, the nested shift ladder, and the two W32 additions equal the certified pure update:

$$\mathsf{generatedUpdate}(x, s) = \mathsf{W32.ofInt}\big(\mathsf{hbzFastEncodeStep}(\mathsf{uint}(x), s)\big).$$

The proof discharges the thirteen concrete symbol cases once in that theory. Erased declassification and `protect` calls are handled according to the extracted semantics; they introduce no new assumption.

One-symbol transition and failure branch

The compiled success transition combines

$$\begin{aligned}\mathsf{encodeTrace}(s :: tail).\mathsf{state} &= \mathsf{fastStep}(\mathsf{normalize}(x_0, s), s),\\ \mathsf{encodeTrace}(s :: tail).\mathsf{bytes} &= \mathsf{normBytes}(x_0, s) \mathbin{+\!\!+} \mathsf{encodeTrace}(tail).\mathsf{bytes}\end{aligned}$$

with the actual inner exit, word update, cursor arithmetic and prefix frame. It reconstructs the complete outer live invariant for index j.

If the actual body finds $off < 4$, it writes $bad = 1$. The loop invariant then deliberately stops requiring a trace equality. The final theorem retains this genuine failure disjunct instead of assuming encoder success.

Whole outer-loop closure

The live outer invariant is

$$\begin{aligned}0 \le i \le 1024,\\ \mathsf{uint}(x) = \mathsf{encodeTrace}(\mathsf{symbolSuffix}(symbols_0, i)).\mathsf{state},\\ \mathsf{uint}(off) + |\mathsf{encodeTrace}(\mathsf{symbolSuffix}(symbols_0, i)).\mathsf{bytes}| = 1024,\end{aligned}$$

together with exact segment matching, an initial-array prefix frame, and $4 \le \mathsf{uint}(off)$. It initializes at $i = 1024$ with state 2^{23}, empty trace, and cursor 1024. A live outer-loop exit implies $i = 0$, so the accumulated suffix belongs to $\mathsf{symbolListOfArray}(symbols_0)$.

Final serialization prepend

The generated procedure subtracts four from off and performs four little-endian byte stores. The compiled finalization theorem turns their weakest-precondition expression into

$$\mathsf{serialize32LE}(\mathsf{uint}(x)) \mathbin{+\!\!+} \mathsf{encodeTrace}(\mathsf{symbolListOfArray}(symbols_0)).\mathsf{bytes} = \mathsf{traceBytes}(\mathsf{symbolListOfArray}(symb$$

It simultaneously preserves the initial prefix below the returned offset.

Compiled generated-procedure theorem

The exact precondition of `actual_rans_encode_trace_closure` is

- initial encoder, state and symbol arrays are named snapshots;
- the concrete mode-2 HBZ encoder table is used;
- (count=1024); and

- the actual symbol array satisfies the mode-2 canonical stream predicate.

Its postcondition is

$$bad \neq 0 \ \vee \ \left(\begin{array}{l} bad = 0 \ \wedge \ 0 \leq off \leq 1020 \ \wedge \ 4 \leq 1024 - off \leq 1024, \\ \mathsf{segmentMatches}(encp, off, \mathsf{traceBytes}(\mathsf{symbolListOfArray}(symbols_0))), \\ off + |\mathsf{traceBytes}(\mathsf{symbolListOfArray}(symbols_0))| = 1024, \\ \mathsf{prefixFrame}(enc_0, encp, off) \end{array} \right).$$

The corollary `actual_rans_encode_trace_refinement` exposes this full suffix result without an existential trace and without success in the precondition. Both theories fresh-compile with `-no-eco`.

Rejected tactics and proof decomposition

A one-shot `inline; sim` was not retried: reverse outer iteration and a variable-length inner loop do not preserve the needed pure continuation automatically. Expanding the generated reciprocal shift ladder inside the outer proof created an unmanageable expression; the dedicated generated-word theorem was rewritten instead. The initial W64 cursor was normalized under its canonical range explicitly. No observation wrapper, procedure copy, arbitrary trace witness, success premise, authored axiom, or 1024-fold unrolling was added.

Partial correctness, non-vacuity, and next gate

The theorem is a Hoare partial-correctness result: every terminating execution satisfies the displayed failure/success disjunction. It does not prove local encoder losslessness, termination, or a terminating all-6 success witness. Thus `OBL-RANS-ACTUAL-SUCCESS-WITNESS` stays PARTIAL.

Claim	Week 11 status
`OBL-RANS-ACTUAL-INNER-NORMALIZE`	PROVED
`OBL-RANS-ENCODE-REFINEMENT`	PROVED/PARTIAL CORRECTNESS
`OBL-RANS-ACTUAL-SUCCESS-WITNESS`	PARTIAL
`OBL-RANS-DECODE-REFINEMENT`	PARTIAL
`OBL-RANS-CORE-INVERSE`	PARTIAL
`OBL-SIG-HBZ-ENCODE-DECODE`	PARTIAL

No decoder success, HBZ round trip, canonical parse, encoding delta, or security result is claimed. The Week 12 decision is GO-ENCODER: the single next obligation is the actual generated decoder trace refinement.

28 Week 12: Actual Decoder Semantic Refinement

Week 12 closes `OBL-RANS-DECODE-REFINEMENT` as PROVED, EXACT-TRACE PARTIAL CORRECTNESS. The target is the actual generated `RansDecodeTarget.M._rans_decode`; it is not a wrapper, observer, or pure decoder theorem.

Claim boundary and theorem surface

The compiled theorem is `actual_rans_decode_trace_refinement` in `Mode2RansDecoderTopHoare`. Its precondition binds the actual generated decoder state and buffer to a canonical exact trace:

- `symsp = decoded0`;
- `statep = state0`;

- `bufp = buffer0`;
- the literal mode-2 `symbol_words` and `dsyms_words` tables;
- `actual_mode2_decoder_trace_input expected_symbols buffer0 state0 encoded_size`.

The exact-trace premise contains the canonical 1024-symbol stream, the exact `traceBytes` buffer, the encoded-size relation $encoded\_size = |\mathsf{traceBytes}(\mathsf{symbolListOfArray}(expected\_symbols))|$, and the concrete state fields $(count, size, m) = (1024, encoded\_size, 13)$. It does *not* contain decoder success, output equality, or a post-state observation.

The postcondition published by the theorem is:

$$bad = 0 \wedge off = encoded\_size \wedge decoded[0..1024) = expected\_symbols[0..1024) \wedge decoded[1024..2048) = deco$$

The internal final-state fact $x = 2^{23}$ is proved at the generated loop exit and used to discharge the actual final reject checks, but it is not exported as an ABI result.

Reference state, cursor, and 0/1/2-byte normalization

For $syms = \mathsf{symbolListOfArray}(expected\_symbols)$, the reference decoder boundary is given by:

$$state(i) = \mathsf{decoderStateAt}(syms, i), \qquad cursor(i) = \mathsf{decoderCursor}(syms, i).$$

The compiled cursor lemmas establish

$$cursor(0) = 4, \qquad cursor(i{+}1) = cursor(i){+}|\mathsf{segment}(i)|, \qquad cursor(1024) = |\mathsf{traceBytes}(syms)|,$$

and $state(0)$ is the parsed little-endian head while $state(1024) = 2^{23}$. The concrete mode-2 bound keeps each normalization segment at length 0, 1, or 2 bytes.

The actual inner invariant tracks the partial replay cursor $k = off - cursor(i)$ and the current reduced state. The generated guard matches the pure replay relation: when one more segment byte remains, the loop consumes it; otherwise the no-consume exit establishes the next state/cursor boundary. This is the exact 0/1/2-byte case split used by the theorem `decoder\_inner\_trace\_step`, `decoder\_inner\_trace\_stop`, and `decoder\_inner\_trace\_exit\_to\`

Concrete table bridge

The word-step bridge is split from the loop proof. The generated code uses the packed `symbol_words` lookup on $x \,\&\, 1023$, stores the recovered symbol, loads the packed `dsyms_words` entry, and computes the reduced state with the exact mode-2 arithmetic. The compiled lemma family `generated\_decoder\_lookup\_word\_from\_mode2`, `generated\_decoder\_symbol\_from\_mode2`, and `generated\_decoder\_word\_update\_from\_mode2` connects those exact generated expressions to the mathematical decoder step without introducing a table axiom.

Exact exit and final checks

At outer-loop exit, `decoder\_outer\_trace\_exit\_components` proves

$$i = 1024, \qquad x = 2^{23}, \qquad off = encoded\_size, \qquad bad = 0,$$

together with the full decoded-prefix equality and the untouched $[1024, 2048)$ output frame. Those facts discharge the generated final-state parse and reject checks. The theorem is therefore exact-trace partial correctness, not losslessness or termination.

Non-vacuity and remaining edge

The exact-trace premise is satisfiable as a pure trace relation and is the mathematical relation produced by the Week 11 encoder success theorem. It does not prove that the actual encoder success branch is reachable. `OBL-RANS-ACTUAL-SUCCESS-WITNESS` therefore remains PARTIAL.

The remaining implementation edge is now only the unary harness composition: transport the encoder suffix and the actual copy-helper slice facts into the decoder's exact pointwise trace-read premise. Until that theorem compiles, `OBL-RANS-CORE-INVERSE` and the HBZ parent remain PARTIAL.

Rejected tactics and follow-up

A monolithic `inline; sim` proof was rejected because the outer symbol loop and the variable-length normalization loop require different invariants. Extending the older control theorem was also rejected: it can only retain `bad` $\in \{0, 1\}$ and cannot express symbol recovery. The Week 13 single edge is the actual encoder→copy→decoder harness theorem; no new codec layer should be introduced before it compiles.

29 Week 13: Actual rANS Core Composition

Week 13 closes `OBL-RANS-CORE-INVERSE` as PROVED, SUCCESS-CONDITIONED PARTIAL CORRECTNESS. The result is a direct composition theorem over three actual extracted procedures, not a production full-HBZ wrapper refinement.

Actual execution path and theorem interface

The existing `Mode2RansActualInverse` harness executes:

`RansEncodeTarget.M._rans_encode`
↓
`HbzFullEncodeTarget.M.__copy_encoded_suffix`
↓
`RansDecodeTarget.M._rans_decode`.

The copy and decoder calls occur only on the actual encoder's zero- `bad` branch. The compiled theorem `actual_rans_encode_copy_decode_inverse` is a Hoare theorem over `Mode2RansActualHarness.run`; it does not replace any of these calls with a pure or observation procedure.

Its complete semantic precondition is exact initial argument binding and `mode2_hbz_symbol_streamexpected_symbols`. In particular, it does not assume encoder success, decoder reachability, copied trace equality, decoder success, or decoded-symbol equality.

Encoder–copy interface

On actual encoder success, the Week 11 theorem supplies

$$\mathsf{segmentMatches}(encoded, off, trace) \quad \wedge \quad off + |trace| = 1024,$$

where $trace = \mathsf{traceBytes}(\mathsf{symbolListOfArray}(symbols))$. The actual copy theorem supplies $\mathsf{sliceEq}(encoded, copied, off, |trace|)$. The Week 13 lemma `copied_suffix_is_exact_trace` proves their pointwise composition:

$$\mathsf{segmentMatches}(copied, 0, trace).$$

Thus the decoder buffer is the actual post-state of the copy helper; it is not an arbitrary witness.

Global trace to decoder byte reads

The Week 12 decoder invariant phrases normalization reads by symbol index, cursor, and intra-segment offset. The new indexing lemma `trace_bytes_trace_segment_nth` proves that the local byte is the corresponding element of the global trace:

$$\mathsf{trace}[\mathsf{cursor}(i) + k] = \mathsf{segment}(i)[k].$$

Combining this fact with the copied global segment yields `segment_matches_implies_exact_decoder_segment_input`. Its W64 corollary feeds every generated byte read required by the actual decoder theorem. No separate pointwise-read assumption remains in the harness precondition.

W64/int size and configured state

The actual harness computes

$$encoded\_size = \mathsf{W64.of\_int}(1024) - encoder\_off.$$

From the actual successful encoder bounds $0 \leq \mathsf{uint}(off) \leq 1020$ and $\mathsf{uint}(off) + |trace| = 1024$, `encoder_success_size_word_bridge` establishes

$$\mathsf{uint}(encoded\_size) = |trace|, \qquad 4 \leq |trace| \leq 1024,$$

and the exact W64 word equality. The proof uses unsigned subtraction under a proved order, rather than identifying modular and integer subtraction implicitly.

The harness then performs three actual array stores. The operation `configured_decoder_state` denotes exactly that store sequence, and `configured_decoder_state_fields` proves the resulting fields are $(count, size, m) = (1024, |trace|, 13)$. Together with the copied segment, this constructs the complete Week 12 `actual_mode2_decoder_trace_input` predicate.

Failure/success disjunction

The final theorem preserves both actual branches:

$$\begin{aligned}
encoder_bad \neq 0 \quad &\Longrightarrow \neg decoder_ran \ \wedge \ decoded = decoded_0 \ \wedge \ state = state_0, \\
encoder_bad = 0 \quad &\Longrightarrow decoder_ran \ \wedge \ decoder_bad = 0 \ \wedge \\
&\quad decoder_off = encoded_size \ \wedge \\
&\quad decoded[0..1024) = symbols[0..1024) \ \wedge \\
&\quad decoded[1024..2048) = decoded_0[1024..2048).
\end{aligned}$$

The success conclusion is obtained by sequentially applying the Week 11 encoder theorem, the actual copy theorem, and the Week 12 decoder theorem. The generated loops are not unfolded again.

Partial correctness, non-vacuity, and rejected approaches

The canonical-stream premise has a compiled all-zero array witness, so the harness precondition is satisfiable. The postcondition is nevertheless a failure/success disjunction. It does not establish that the success branch is reachable, nor does it establish encoder or decoder termination/losslessness. `OBL-RANS-ACTUAL-SUCCESS-WITNESS` therefore remains PARTIAL.

A monolithic `inline; sim` proof was rejected because it duplicates the completed encoder and decoder semantic invariants. Supplying copied-trace or decoder-read equality as a top-level premise was rejected as circular, and constructing a decoder buffer witness was rejected because it severs the actual copy-helper path. The adopted decomposition uses only explicit array-index, W64/int, and decoder-state adapters between the compiled actual procedure theorems.

Security boundary and Week 14 gate

This theorem is not the production `_encode_hb_z1_full`/`_decode_hb_z1_full` inverse. It proves neither malformed-input rejection nor canonical parsing and does not justify an encoding zero-loss or EUF-CMA claim. Accordingly `OBL-SIG-HBZ-ENCODE-DECODE` remains PARTIAL.

The Week 14 single gate is a success-conditioned composition at the two actual full-HBZ wrappers, using the already proved prepare/apply inverse and the new rANS core inverse. Expansion to the h codec remains prohibited until that wrapper theorem fresh-compiles.

30 Week 14: Actual Full-HBZ Wrapper Composition

Week 14 closes `OBL-SIG-HBZ-ENCODE-DECODE` as PROVED, SUCCESS-CONDITIONED HOARE PARTIAL CORRECTNESS. The proof retains the actual encoder's zero-size failure branch; encoder success is not a theorem premise.

Internal extraction boundary

The full-wrapper extraction and the focused Week 8–13 extractions place four internal procedures in different EasyCrypt modules. The theory `Mode2HbzInternalBoundaries` proves exact equivalences for `_encode_hb_z1_prepare`, `_rans_encode`, `_rans_decode`, and `_decode_hb_z1_apply`. Each proof is a small `proc; sim` argument over the actual extracted bodies. The focused prepare, encoder, decoder, and apply Hoare theorems are then lifted through these equivalences; no generated loop is re-proved.

Two semantic adapters connect the lifted results. For a canonical mode-2 HBZ array, `prepared_hbz_implies_mode2_symbol_stream` turns the actual prepare prefix into the canonical rANS symbol-stream predicate. After the actual decoder, `decoded_prefix_preserves_prepared_hbz` transports symbol-prefix equality back to the prepare predicate required by the actual apply theorem.

Actual full encoder and decoder

The theorem `actual_encode_hb_z1_full_mode2_trace` targets `HbzFullEncodeTarget.M._encode_hb_z1_fu` directly. Its precondition binds the initial arrays and literal mode-2 table, count (1024), alphabet (13), offset (6), and a canonical HBZ input. Its postcondition is the following disjunction:

$$
\begin{aligned}
&size = 0 \quad \wedge encoded = encoded_0,\\
&size \neq 0 \quad \wedge\ 4 \le \mathsf{uint}(size) \le 1024\\
&\qquad\quad \wedge\ size = \mathsf{W64.of\_int}(|trace(prepared)|)\\
&\qquad\quad \wedge\ \mathsf{segmentMatches}(encoded, 0, trace(prepared))\\
&\qquad\quad \wedge\ \mathsf{suffixFrame}(encoded_0, encoded, \mathsf{uint}(size)).
\end{aligned}
$$

In both proof branches the prepared-symbol witness comes from the actual prepare call. Canonicality excludes prepare failure; the zero-size return is therefore the preserved actual rANS failure path, not a claimed unreachable case.

The theorem `actual_decode_hb_z1_full_mode2_inverse` targets `HbzFullDecodeTarget.M._decode_hb_` directly. Given the exact successful trace produced by the encoder theorem, it proves decoder `bad`=0, recovery of coefficients (0...1023), and preservation of the initial coefficient tail (1024...2047). The proof follows the generated control flow: the actual apply call occurs only after the actual rANS decoder establishes its zero-bad state. Apply execution and coefficient equality are conclusions, not premises.

Direct pair and production lift

`HbzFullActualHarness.run` directly calls the two focused full wrappers. It observes the returned size, skips decoding when the size is zero, and otherwise supplies the returned encoded buffer and size to the actual full decoder. The compiled theorem `actual_hbz_full_encode_decode_inverse_m` assumes only exact caller bindings and canonical HBZ coefficients. Its failure branch returns size zero, records that the decoder did not run, and preserves the encoded, decoded, and bad arrays. Its success branch records the size bounds, actual prepare trace, decoder execution and zero-bad result, coefficient-prefix inverse, coefficient-tail frame, and encoded-output tail frame.

`Mode2HbzSignatureBoundaryLift` repeats this direct call shape at `SignaturePackMode2Target.M._encode` and `SignatureUnpackMode2Target.M._decode_hb_z1_full`. A compiled exact equivalence connects the production harness to the focused harness, using the already checked production/focused procedure identities. Consequently `signature_pack_unpack_hbz_full_inverse_mode2` has the same failure/success contract at the signature extraction boundary.

Claim boundary

These are unary Hoare partial-correctness results. They do not prove termination or losslessness, that every canonical input reaches encoder success, or that the zero-size branch is unreachable. The optional all-zero actual success witness remains PARTIAL. Malformed-input rejection, canonical parsing, the (h) codec, metadata and padding, public-context mu, product/replay, complete Sign/Verify correctness, encoding zero-loss, and EUF-CMA security remain outside Week 14.

The verification pipeline fresh-compiles 72 authored targets with `-no-eco`, regenerates both focused and signature extractions, checks their procedure identity and hashes, regenerates the deterministic HBZ table certificate, scans proof holes and authored axioms, checks selected upstream baselines and read-only roots, and builds these notes without LaTeX errors.

31 Week 15: Fixed All-Six Actual Encoder Success

Week 15 closes `OBL-RANS-ACTUAL-SUCCESS-WITNESS` as PROVED FOR THE FIXED ALL-SIX INPUT, HOARE PARTIAL CORRECTNESS. The result excludes the actual encoder's failure return for every terminating run from the fixed precondition. It does not establish termination or a probability-one reachability theorem.

Zero HBZ to the actual all-six prepare result

`Mode2RansAllSixBudget` defines `zero_hbz` by initializing every byte of the 8192-byte coefficient array to zero. The lemma `zero_hbz_get32` opens the four-byte `BArray8192.get32` load and proves every coefficient word is `W32.zero`; canonicality is then derived by `zero_hbz_canonical`. Thus canonicality is not an input assumption of the final witness theorem.

The actual focused `_encode_hb_z1_prepare` theorem yields `bad=0`, the prepared-prefix relation, and its byte-tail frame. `prepared_zero_hbz_is_all_six` unfolds each prepared coefficient, uses the zero word load, and proves that all first 1024 symbols equal 6. Consequently `actual_focused_encode_hb_z1_prepare_zero_hbz` connects the concrete zero coefficient input to the all-six predicate without an arbitrary prepared-symbol witness.

A coarse capacity proof

The literal table certificate gives

$$f(6) = 398, \qquad x_{\max}(6) = 2{,}097{,}152 \cdot 398 = 834{,}666{,}496.$$

For every state in the actual encoder range $2^{23} \le x < 2^{31}$, integer division gives $x \operatorname{div} 256 < x_{\max}(6)$. The lemma `symbol6_normalization_len_le1` therefore proves $0 \le \ell_6(x) \le 1$: symbol 6 emits at most one normalization byte.

The first four all-six suffix states are checked inside EasyCrypt by unfolding the certified fast/math update:

$$2^{23},\ 21{,}582{,}496,\ 55{,}528{,}910,\ 142{,}868{,}116,\ 367{,}580{,}518.$$

The first four inputs are below $x_{\max}(6)$ before their updates, so `all_six_first_four_no_normalization` proves zero emitted bytes for each of these four prefixes. Induction then combines four zero-emission steps with at most one byte for every remaining step:

$$|\mathsf{encodeTrace}(6^n).\mathsf{bytes}| \le \max(0, n-4).$$

At $n = 1024$, `all_six_trace_fits_mode2` derives at most 1020 normalization bytes and a total serialized trace size between 4 and 1024. No externally computed exact trace length is used as proof evidence.

Failure guard and actual success

The strengthened Week 11 closure retains the actual failure cause rather than only `bad` $\neq 0$. The theorem `actual_rans_encode_failure_trace_cause` proves that a nonzero bad return requires more than 1020 normalization bytes. This certificate is threaded from the generated `off < 4` guard through the existing outer/inner tail invariants; the encoder loops are not copied or re-proved in a new theory.

`actual_rans_encode_all_six_success` targets `RansEncodeTarget.M._rans_encode` directly. Its precondition binds the actual arrays, literal mode-2 table, count 1024, and the all-six prefix; it does not assume success, buffer fit, trace size, or a returned offset. The pure 1020-byte upper bound contradicts the actual failure certificate. The remaining branch has `bad=0`, offset in $[0, 1020]$, serialized size in $[4, 1024]$, the exact all-six trace segment, and the initial prefix frame.

Full and production boundaries

`actual_encode_hb_z1_full_zero_success` opens the actual `HbzFullEncodeTarget.M._encode_hb_z1_full`. It derives zero-input canonicality, prepare success, and all-six symbols internally; calls the actual rANS encoder theorem; and follows the actual suffix-copy branch. Its postcondition gives a nonzero returned size, the 4–1024 bound, exact trace, output-tail frame, and the actual prepare-call symbol witness.

The exact focused/production procedure equivalence yields `signature_pack_hbz_zero_success_mode2` for `SignaturePackMode2Target.M._encode_hb_z1_full`. The additional corollary `signature_pack_unpack_` eliminates the failure disjunct of the existing production pack/unpack harness using the new encoder theorem, then obtains decoder execution, decoder `bad=0`, the zero coefficient prefix, and both coefficient and encoded tail frames.

Boundary and next target

All Week 15 claims remain unary Hoare partial correctness. No `phoare`=1 or losslessness theorem was compiled, so fixed-input termination and actual non-vacuous reachability remain PARTIAL. General encoder termination, all-canonical-input success, malformed-input rejection, encoding zero-loss, and security are not claimed. The Week 16 single recommended target is `OBL-SIG-H-ENCODE-DECODE`: the actual mode-2 h-suffix codec leaf/table/loop/full-wrapper inverse.

32 Week 16: MINCORE KeyGen First Task

The Week 16 first task originally exited as CONTINUE-KG: BLOCKED-KG-NTT-MUL. The continuation reported in the next section supersedes that intermediate decision with STOP-KG-NTT, not GO-KG. A transparent Hoare harness now composes the actual mode-2 `_kp_m23_matrix` and `_keypair_finalize_m23` procedures. The faithful paper claims (KG-1)–(KG-4) and $As = qj \pmod{2q}$ remain incomplete because the checked matrix output has not yet been related to the security model's polynomial multiplication.

Baseline and actual boundary

Before modification, the aggregate verifier fresh-compiled 74 authored targets with `-no-eco`. The preserved log is `logs/verify-all-before-week16.log`, with SHA-256 `da7a7516166d54e93665d36e9a813`

`Mode2KeygenCoreEquation.ActualM23MatrixFinalizeSnapshot.run` first invokes the generated parent procedure `_kp_m23_matrix` with $(rows, cols) = (2, 3)$, records the returned pre-finalization `bp` array, and then invokes `_keypair_finalize_m23` with count 512. It returns the actual snapshot, transformed-secret scratch, finalized high-part array, and adjusted `s2`. The harness contains no sampler, retry loop, packer, acceptance guard, or public API.

Compiled snapshot consequences

The Hoare precondition binds all six initial arrays and requires only the existing matrix bound, the three input-polynomial representations, centered active `s2` words, and canonical active `avec` words. It does not assume finalizer success, a KG predicate, or a desired equation.

`actual_m23_matrix_finalize_snapshot` exports the actual matrix output and NTT-scratch representations, exact finalizer output, and tail frames. The stronger `actual_m23_matrix_finalize_semantic` additionally exports the scalar and HAETAE finalizer semantics. For every active coefficient, if

$$r = (pre\_bp + e + a) \bmod q, \qquad b_0 = \mathsf{low}(r),$$

then the compiled result includes

$$r = 2b_1 + b_0, \qquad -1 \le b_0 \le 1, \qquad e' = e - b_0.$$

This is the KG-2 coefficient decomposition and the adjusted component of KG-4 at the actual finalizer boundary.

`Mode2KeygenSnapshotAlgebra` proves the required integer division, parity, and reduction lemmas internally. The resulting actual predicate is the deliberately snapshot-only congruence

$$2(a - 2b_1) + 2\,pre\_bp + 2e' \equiv 0 \pmod{2q}.$$

The name `actual_snapshot_mod2q_zero` preserves that distinction; the returned snapshot is not silently renamed to $A_{\mathrm{gen}} s_{\mathrm{gen}}$.

Exact blocker

The checked matrix postcondition represents each returned row by

$$\mathsf{output\_row} = \mathsf{array256\_mont}(\mathsf{full\_invntt}(\mathsf{pointwise\_row\_ntt}(\cdots))).$$

No compiled theorem identifies this object with the security model's row product $\sum_c A_{\mathrm{gen}}[r, c] s_{\mathrm{gen}}[c]$. The legacy `Rq.ntt` cannot be substituted: the existing full-NTT specification contains a theorem showing that the two transforms differ on one.

Consequently KG-1 and KG-3 are BLOCKED; KG-4 is partial because only the adjusted $e - b_0$ component is constructed; and the paper $As = qj \pmod{2q}$ conclusion is BLOCKED. The claim ledger keeps `OBL-MINCORE-KEYGEN` partial. Sampler distributions, retry termination, packing, public APIs, Sign/Verify correctness, and security are outside this result.

Historical continuation target

The requested KeyGen continuation was one representation theorem: connect `KeygenM23ArithmeticSpec.outp` to the security model's $A_{\mathrm{gen}} s_{\mathrm{gen}}$ polynomial product through the checked full NTT. The next section records why that bridge did not close and why the active lane moves to Sign without assembling KG-1, KG-3, complete KG-4, or the paper key equation.

33 Week 16 Continuation: KG–NTT–MUL Stop Boundary

The continuation exits as STOP-KG-NTT, not GO-KG. The actual `_kp_m23_matrix` chain already has checked procedure theorems for the forward transform, three-column pointwise accumulation, and inverse transform. A new authored boundary theorem fresh-compiles the final rewrite to

$$\mathsf{array256\_mont}(\mathsf{full\_invntt}(\mathsf{pointwise\_row\_words}(\cdots))),$$

but no checked theorem identifies this value with the negacyclic row product $A_{\mathrm{gen}} s_{\mathrm{gen}}$.

The compiled corollary `actual_m23_matrix_snapshot_rows_explicit` applies the rewrite to both active rows of the transparent harness that directly calls `_kp_m23_matrix` and `_keypair_finalize_m23`. Its precondition contains only the six initial-array bindings, matrix/input representation bounds, centered active s_2, and canonical active a. Its postcondition exposes each returned row slice as $\mathsf{array256\_mont}(\mathsf{full\_invntt}(\mathsf{pointwise\_row\_words}(\cdots)))$. It is a consequence of the existing snapshot theorem, not a rewritten loop proof, and it assumes no product or key equation.

Exact missing leaf

For one matrix spectrum $\widehat{a}$ and one coefficient-domain secret polynomial p, the first missing theorem is

$$\mathsf{mont}\left(\mathsf{full\_invntt}\left(\left[\widehat{a}_j\,\mathsf{full\_ntt}(p)_j R^{-1}\right]_{j=0}^{255}\right)\right) = \mathsf{full\_invntt}(\widehat{a}) * p,$$

where $*$ is multiplication in $\mathbb{F}_q[X]/(X^{256}+1)$. A direct expansion needs the absent odd-root orthogonality formula

$$\sum_{j=0}^{255} \zeta^{(2\,\mathrm{br}(j)+1)(k-i)} = \begin{cases} 256, & k = i, \\ 0, & k \neq i, \end{cases} \qquad 0 \le i, k < 256.$$

The checked NTT development proves each executable transform against its closed formula, but does not contain this orthogonality lemma, a standalone $\mathsf{full\_invntt}(\mathsf{full\_ntt}(p)) = p$ theorem, or the pointwise-product/convolution theorem. The legacy incomplete `Rq.ntt` is unusable because a checked counterexample distinguishes it from `full\_ntt`.

After the native array theorem, a second adapter must preserve negacyclic multiplication while mapping Rq.poly field arrays to the integer-list polynomials used by `HAETAE_Algebra.poly_mul` and `matrix_vec_mul`. That adapter is also absent.

Frozen KeyGen claim and transition

KG-2 remains proved at the actual finalizer boundary: $b = b_0 + 2b_1$, $b_0 \in \{-1, 0, 1\}$. The adjusted words $e' = e - b_0$ remain a partial KG-4 component. KG-1, KG-3, the complete KG-4 object, and $As = qj \pmod{2q}$ are not proved. In particular, the existing snapshot congruence over `pre_bp` is not relabelled as the paper key equation.

The KeyGen lane is now frozen at this finalization boundary. The next single MINCORE target moves to the actual accepted Sign helpers `_sf_round_challenge_mode2`, `_sf_z_check`,

and `_sf_hint_mode2`. Sampler distributions, retry termination, packing, public APIs, and general NTT algebra remain outside that target.

The final single-instance verifier run fresh-compiled all 77 authored targets with `-no-eco`, passed hole/axiom/debug, extraction/table drift, selected baseline, LaTeX, and read-only-root checks, and ended with `RESULT PASS authored-targets=77 cache=-no-eco`. An independent read-only verifier audited the actual-call and precondition surfaces and returned PASS for this stop decision.

34 Week 16 MINCORE–SIGN: Actual Control and Stop Boundary

The Sign time box exits as STOP-SIGN-CHAL-MODE2, not GO-SIGN. A focused extraction selects exactly the actual `_sf_round_challenge_mode2`, `_sf_z_check`, and `_sf_hint_mode2` roots. The transparent `ActualSignAcceptedCore.run` harness calls them in that order, with the hint helper guarded by the actual zero rejection result.

The fresh-compiled theorem `actual_sign_accepted_core_branch_control_mode2` has precondition `true` and proves that the recorded accepted branch is equivalent to the returned `_sf_z_check` word being zero. It does not assume or prove rejection success, a response equation, a norm predicate, a hint equation, or final output equality. The harness includes neither the sampler nor a retry loop, packer, codec, or public API.

Exact missing semantic leaf

The first Sign-specific absent theorem is `sf_challenge_mode2_highbits_lsb_sampleinball_correct`. It must connect the actual high-bit calculation and packing, the packed LSB of the rounded y_0, and the first 32 bytes of μ to the mode-2 $\mathsf{SampleInBall}(\cdots, 58)$ challenge. Existing checked transcript work stops at the μ-absorb seam. Moreover, the current abstract security model's challenge operator omits its highbits input, so it is not a sound replacement for this leaf.

Paper-literal S-1 and S-4 independently retain the already-frozen full-NTT pointwise-product/convolution dependency. S-4 also needs a checked response representation theorem; S-5 and S-6 need signed-word, no-wrap accumulator and strict-comparison semantics for `_sf_add_cs_and_check`; S-7 needs the fixed-point rounding/highbits interpretation of `_sf_hint_mode2`. Those source-level calculations are not promoted to proved equations.

The 77-target pre-Sign aggregate was preserved before this change. The new manifest adds one focused authored target without replacing any baseline target. Sampler distributions, Sign termination, KeyGen NTT expansion, packing, codecs and public APIs remain outside scope. Under the stop rule, the active MINCORE lane moves to the canonical decoded-object Verify core.

References

[1] CryptoLab. Cryptolab haetae specification pdf, 2026. Pinned local artifact, SHA-256 80d28b3af9af2a27dba0eb4746f9b5755e05cb9a86bb6596f45fda06c605f3f6.

[2] CryptoLab. Existing haetae easycrypt refinement tree, 2026. Local source root haetae-ref-easycrypt/ reused through import-only adapters.

[3] CryptoLab. Pinned haetae jasmin implementation tree, 2026. Local source root haetae-ref-jasmin/ used as the extraction target.